Cloud-Offload Compute Architecture — Master Spec (v1.1)

Cloud-Offload Compute Architecture — Master Spec
1. Executive Summary
2. Goals & Non-Goals
3. Locked Architectural Decisions
4. Architecture Overview
5. Schema State
6. Wire Contracts
7. Repo Layout (target end state)
8. Migration Sequencing
9. Implementation Rows
   Phase A — Foundation
   Phase B — First Lambda end-to-end (proves the pattern)
   Phase C — Heavy compute Lambda + orchestrator changes
   Phase D — Cancel + Fail semantics
   Phase E — Visibility + history
   Phase F — Remaining workloads
   Phase G — Closeout
10. Future Ladder (v2 / v3 — explicitly NOT BUILT)
11. Verifications & Smoke-Testing Protocol
   11.1 Cross-cutting one-time verifications (before C-rows)
   11.2 Smoke-testing protocol
   11.3 Brainstorm-first vs writing-plans-direct
   11.4 Smoke session execution protocol
   11.5 Post-row completion protocol
   11.6 Pre-row-start protocol
12. Tracking Protocol
13. Cold-Session Prompt Templates
   13.1 Implementation cold-session prompt
   13.2 Smoke-test cold-session prompt
14. Doc Kill List
15. Changelog

Cloud-Offload Compute Architecture — Master Spec

Status: Locked (2026-05-03). Living document — updated per row close. See §15 changelog for amendments. **Version:** v1.1 (2026-05-03 — deployment topology amendment; smoke protocol reality-aligned) **Owner:** Arun (engineering authority) +

Prasanta (solar-domain authority). **Supersedes:** `docs/post-parity/PRD-cable-compute-strategy.md`, `docs/initiatives/2026-05-01-cable-compute-offload-feasibility.md`, the architectural framing of `docs/initiatives/findings/2026-05-02-002-refund-on-cancel-policy.md` §B.6, and the halted plan at `docs/superpowers/plans/2026-05-02-b32-failed-runs-path.md`. See §14 for the full disposition. **Cross-references:** `docs/initiatives/post-parity-v2-backend-plan.md`, `docs/PLAN.md`, `CLAUDE.md`.

> **Cold-session reader:** if you arrived here via a fresh Claude Code session, skip to §13 for the prompt that launched you, then read this document end-to-end **before any code action**. Locked decisions (D1–D23) are non-negotiable; row scopes (C1–C21) are bounded; reality wins over spec on the verifications.

---

1. Executive Summary

We are moving the entire heavy compute path of SolarLayout off the user's machine and into AWS. The Python sidecar (`python/pvlayout_engine/`) dies; its domain-knowledge core (`pvlayout_core/`) lives on as a standalone library packaged into Lambda container images. The Tauri desktop becomes a thin HTTP+UI client identical in shape to the future Expo/React-Native mobile app. Heavy compute runs in SQS-triggered Lambdas that write directly to RDS via psycopg2; fast operations run in synchronously-invoked Lambdas. Compute artifacts (layout JSON + DXF + PDF + KMZ + thumbnail) are pre-rendered together inside one Lambda invocation per Run; download buttons are pure presigned-GET URL fetches.

This is the architectural reset that unblocks mobile, retires the PyInstaller bundling burden, gives us a single execution path and a single engine-version source of truth, and provides honest horizontal-scale headroom under SQS at-least-once semantics. The trade is ~3-15s of network latency on operations that are instant locally today, accepted in exchange for one engine, one contract, one form-factor-agnostic compute path.

2. Goals & Non-Goals

Goals (v1): - Single execution path for heavy compute, runnable from desktop and mobile clients identically. - Sidecar deletion as a first-class outcome (not a vague "eventually"). - Clean cancel + fail + refund semantics with `Run.status` as the single lifecycle truth. - Retry-safe at every layer: SQS at-least-once + idempotent UPDATE patterns + DLQ from day 1. - Pre-paid-pack reliability — never lose a calc to infrastructure flakiness.

Non-goals (v1, explicitly): - Per-plot fan-out (`Slice` table). Future ladder only — see §10. - Functional split of energy from layout into separate Lambdas. Future ladder only. - Fargate / ECS deployment. Same Docker image is portable to ECS via `CMD` change; not built. - IaC (Terraform / CDK). Manual `aws` CLI provisioning is acceptable v1. - Server-side rate limiting. Client-side debounce + idempotency is sufficient. - DLQ replay tooling in mvp_admin. Manual-CLI replay is acceptable v1.

- Wasm / Pyodide client-side parsing. One execution path; KMZ parses in cloud. - Telemetry endpoint (`B22`). Deferred. - Provisioned concurrency on Lambdas. Cold start absorbed by spinner UX.

3. Locked Architectural Decisions

These came out of the brainstorm captured in this session (transcript-level notes superseded by this file). Cite by ID in implementation rows. Spec amendments require a dedicated commit with the changed D-ID and rationale.

- **D1 — Cloud-offload model.** Heavy compute moves to AWS Lambda containers. Tauri and future Expo/RN apps are thin HTTP+UI clients.
- **D2 — Sidecar dies.** `python/pvlayout_engine/` is removed in C19. No sidecar in the end state on any form factor.
- **D3 — Figma-style edits.** Drawing obstructions / adding roads / overriding ICR positions are client-side state mutations on `Project.edits` (autosaved via existing B13). The server only ever receives `(KMZ, edits, params) → fresh Run`. There are no incremental-edit endpoints in the cloud world.
- **D4 — One engine, one knowledge asset.** `pvlayout_core/` is the single source of solar-domain truth. Prasanta is the authoritative reviewer for that surface. Lambdas + handlers + glue are SWE-side.
- **D5 — Lambda Dockerfile `COPY` from monorepo.** No wheel registry. Build context is the repo root; Dockerfiles `COPY python/pvlayout_core` + `pip install` it directly. Image tag = git SHA.
- **D6 — `pvlayout_core/` extracted standalone.** New location: `python/pvlayout_core/` (sibling to the dying `pvlayout_engine/`), own `pyproject.toml`, own `tests/`, own CI gate. C2.
- **D7 — Sync invoke for sub-2s workloads.** Only `parse-kmz` qualifies in v1. mvp_api invokes via `LambdaClient.Invoke({InvocationType: "RequestResponse"})` and blocks on the response.
- **D8 — SQS async for minutes-long workloads.** `compute-layout` and `detect-water` are SQS-triggered. Standard queues (not FIFO). DLQ from day 1 with `RedrivePolicy.maxReceiveCount = 3`. Visibility timeout = Lambda timeout × 1.2.
- **D9 — Lambda direct-to-RDS, no callbacks.** Lambdas hold the RDS connection string in env and write `Run.status` + `UsageRecord` rows directly via psycopg2. There are no internal HTTP callback endpoints, no internal-secret middleware, no `MVP_INTERNAL_SHARED_SECRET`. The halted B32 plan was wrong on this axis; it is deleted in C1.
- **D10 — One Lambda = one image = one package.** No mono-image dispatch. Each `python/lambdas/<purpose>/` is its own deployment artifact with its own dependency graph and memory/timeout config.
- **D11 — Per-Run unit of compute (v1).** One SQS message per Run, one Lambda invocation per Run, one Run row, one S3 layout result blob. No `Slice` table. Per-plot fan-out is v3 (§10), not built.
- **D12 — Pre-render all artifacts inside `compute-layout`.** A single Lambda invocation produces layout JSON + DXF + PDF + KMZ + thumbnail and PUTs all five to S3 at deterministic key paths. Conditional content (energy section, cable layers) is decided by the rendering functions inside `pvlayout_core/exporters/`. There are no separate `export-*` Lambdas.
- **D13 — Energy rolled into `compute-layout` (v1).** Generate Layout is the single user trigger; the Lambda decides whether to compute energy based on

Run params + tier entitlements. **Energy is feature-gated** via the `energy_yield` + `generation_estimates` feature keys (Pro Plus tier per V2 plan §2 + `seed-products.ts`); B16's existing entitlement-debit machinery enforces this at Run-create time. The Tauri Inspector has a separate `Energy` tab that reads `Run.energyResultBlobUrl`; users without entitlement OR runs where energy wasn't requested see an empty/upsell state. **Implementation status note:** `pvlayout_core` already has the energy compute module fully working (parity-tested vs legacy PVlayout_Advance), but the cross-runtime wiring (Lambda branch + desktop Inspector tab + feature-gating-in-UI) is **not yet built** — tracked in C18, which is intentionally brainstorm-first because several practical UX questions aren't nailed down at spec time. Functional split into a separate `compute-energy` Lambda is v2 future ladder, not v1.

- **D14 — Idempotent slice-update SQL.** Lambdas update `Run` with `WHERE status IN ('RUNNING')` so SQS at-least-once redelivery is harmless. Even though there's no `Slice` table v1, the pattern is established for v3 forward-compat.
- **D15 — `Run.status` is the single lifecycle truth.** Lifecycle: `QUEUED` (mvp_api creates) → `RUNNING` (Lambda picks up) → `DONE | FAILED | CANCELLED`. No separate `Job` table. Status enum already supports this post-B29 (RUNNING/DONE/CANCELLED/FAILED — adding QUEUED needs verification, see §11).
- **D16 — Cancel: B30 + Lambda cancel-marker check.** Desktop calls B30 `cancelRunV2` (already shipped). Lambda's completion path does `SELECT … FOR UPDATE` on `Run.status` immediately before flipping to DONE; if status is CANCELLED, abort the upload (best-effort S3 cleanup; orphan tolerable).
- **D17 — Fail: Lambda direct write.** Lambda's exception handler runs the same transactional pattern as the cancel endpoint: `BEGIN → UPDATE Run SET status='FAILED', failedAt=NOW(), failureReason=<text> + INSERT UsageRecord (count=-1, kind='refund', refundsRecordId=<original>) → COMMIT`. No HTTP callback. The B27 memo §B.6 is amended in C1.
- **D18 — Stuck-RUNNING reconciler.** A scheduled job (mvp_api cron, or Vercel Cron, or admin-triggered to start) sweeps Runs in `RUNNING` state older than N minutes (default 30) and flips them to `FAILED` with `failureReason='reconciler:timeout'` + refund row. Catches Lambdas that crash before writing FAILED.
- **D19 — Orchestrator publish-then-commit.** mvp_api opens a DB tx → entitlement debit + UsageRecord (charge) + Run create (status=QUEUED) → SQS SendMessage → COMMIT. SQS failure rolls the tx back. Outbox pattern deferred until observed need.
- **D20 — Lambda credentials via env.** RDS connection string + AWS region in Lambda env vars. Rotate via Secrets Manager when compliance asks. No IAM-database-auth in v1.
- **D21 — Engine version recorded per Run.** `compute-layout` writes the git SHA of its image to a new column `Run.engineVersion: String?` (added in C7's mvp_api migration row). Mid-deploy drift is observable by post-hoc query.
- **D22 — Lambda → ECS portability.** Same Docker image works for ECS via `CMD` change. Documented escape hatch; not built v1.
- **D23 — Compound feature gating for compute Runs.** A single Run may invoke multiple gated compute features: layout (always), cable routing (conditionally, per `params.enable_cable_calc` + `cable_routing` entitlement; works today in Tauri via the LayoutPanel toggle), energy yield (conditionally, per `energy_yield`/`generation_estimates` entitlement; not yet wired — see D13 + C18). Gating enforces at three layers: **(1)** client UI gates

per-feature affordances (toggles, tabs, upsell chips) per the user's `availableFeatures` union — same as today on Tauri; **(2)** mvp_api B16 enforces at Run-create time — for each gated feature implied by `params`, verify the corresponding feature key is in `availableFeatures`; reject 402 if missing; **(3)** the Lambda trusts what's in params — no entitlement re-validation, no DB entitlement query (keeps Lambda simple per D9). **Migration note:** today, layer-2 cable enforcement lives in the sidecar's `require_feature("cable_routing")` dependency on the `/layout` route. When the sidecar dies (C19), that enforcement must already be live in B16. C7 picks this up explicitly. Energy gating follows the same pattern when C18 lands.

4. Architecture Overview

Three first-class API client surfaces (long-run vision):

Surface	App	Auth	Status	Notes
Desktop	`apps/desktop` (Tauri)	license-key bearer	live	Layout app on macOS / Windows / Linux.
Mobile	Expo / React Native (future)	license-key bearer	future	Same wire contracts as desktop; same Lambda compute path.
Web — layout app	`apps/mvp_web` (new route group, future)	license-key bearer (or Clerk JWT bridged)	future	Browser-rendered layout app. **Lives as additional pages inside the existing `mvp_web` Next.js app, not a separate site/app/repo.**
Web — marketing + dashboard	`apps/mvp_web` (existing route groups)	public / Clerk JWT (humans)	live	solarlayout.in. Customer dashboard reads V1 `/entitlements` + `/dashboard/usage` (extended for status badges in C15).
Web — admin	`apps/mvp_admin`	Clerk JWT (ADMIN role)	live	admin.solarlayout.in. Customer + transaction inspection.

All five surfaces are clients of the same `mvp_api` and the same RDS `mvp_db`. Auth distinction is real but bounded: license-key bearer for the *layout-app* surfaces (desktop, mobile, future web layout app) — they act on behalf of an end-user-license-holder; Clerk JWT for the *human-operator* surfaces (mvp_web dashboard, mvp_admin) — they act on behalf of an authenticated human session. The two auth schemes coexist at the mvp_api middleware layer per existing `licenseKeyAuth` and `clerkAuth`.

Compute flow (focuses on the layout-app surfaces; Clerk-authed surfaces are read-only against the same DB):

```
   ┌──────────┐   ┌──────────┐   ┌──────────────┐
┌──────────────┐   ┌──────────┐
   │  Tauri   │   │ Expo/RN  │   │ mvp_web      │         │
mvp_web    │   │ mvp_admin│
   │ desktop  │   │ mobile   │   │ (future:     │         │
marketing  │   │          │
   │          │   │          │   │ layout app   │         │ +
dashboard│   │          │
   │          │   │          │   │ pages)       │
│          │   │          │
   │ (live)   │   │ (future) │   │ (future)     │         │
(live)     │   │ (live)   │
   └──────┬───┘   └──────┬───┘   └──────┬───────┘
└──────┬───────┘   └──────┬───────┘
          │              │               │
│             │
          │  license-key bearer          │                    │  Clerk
JWT  │
          └──────────────┴───────────────┘
└──────┬──────┘
                         │                                       │
                         ▼                                       ▼

┌──────────────────────────────────────────────────────────────┐
                │
mvp_api                            │  Hono/Bun on Vercel
                │  V2 layout-app routes  ·  V1 entitlements
(frozen)     │
                │  /admin/*  ·  /webhooks/
stripe                            │

└────┬──────────────┬──────────────────────────────┬──────────┘
          │              │              │                        │
          └──────────────┘              │                        │
          │                             ▼                        ▼
          ▼                     ┌────────────────┐
┌────────────────┐
┌──────────────┐            │  SQS queue     │         │  RDS
Postgres  │◄──┐
│ parse-kmz    │            │ compute-layout │         │
(mvp_db)      │   │
│ Lambda       │            │  + DLQ         │
└──────────────┘   │
│ (sync invoke)│
└──────┬───────┘                              │
└──────────────┘
│                                     │

▼                                     │
                            ┌────────────────┐    psycopg2: UPDATE
Run       │
```

```
                              │ compute-layout │
──────────────────────────────┘
                              │ Lambda image   │
                              │ (pvlayout_core │
                              │  inside)       │
                              └────────────────┘
                                      │
                                      ▼
                                 ┌─────────┐
                                 │   S3    │  layout.json, exports/
{dxf,pdf,kmz},
                                 │ projects │  thumbnail.webp
                                 │  bucket  │
                                 └─────────┘
```

mvp_web's dashboard and mvp_admin do NOT initiate compute; they read from the same RDS for status, history, and entitlements. The future browser-rendered layout app WILL initiate compute through the same `/v2/projects/:id/runs` flow as desktop and mobile — it's a new client surface, not a new API surface.

Important physical-layout note for future sessions: the future browser layout app is **not a separate Vercel project, app directory, or repository**. It is a new route group inside the existing `apps/mvp_web` Next.js app, sharing the same deploy target, build pipeline, environment variables, and component library as the marketing + dashboard pages. Do NOT scaffold a new top-level `apps/web/` or `apps/mvp_web_layout/` when this surface is built. The two visual boxes for "mvp_web (future: layout app pages)" and "mvp_web marketing + dashboard" in the diagram refer to **two route groups within the same Next.js app**, with different auth schemes per route group.

Generate-Layout flow (canonical):

1. Desktop POST `/v2/projects/:id/runs` (B16) with `{params, edits, idempotencyKey}`.
2. mvp_api opens tx → debit entitlement + UsageRecord(charge) + Run.create(status=QUEUED) → SQS SendMessage to `pvlayout-compute-layout-jobs` → COMMIT (D19).
3. mvp_api returns `{run, ...}` immediately. Desktop starts polling `GET /v2/projects/:id/runs/:runId` (B17) every ~2s.
4. SQS triggers `compute-layout` Lambda. Lambda's first action: `UPDATE Run SET status='RUNNING' WHERE id=$1 AND status='QUEUED'` (idempotent against redelivery, D14).
5. Lambda fetches KMZ from S3 (already there from B6 at project create), runs `pvlayout_core.run_layout(...)`, then renders DXF + PDF + KMZ + thumbnail inline (D12). All artifacts PUT to S3 at deterministic keys.
6. Before flipping DONE, Lambda re-reads `Run.status FOR UPDATE` (D16). If CANCELLED, abort upload + S3 cleanup. Else `UPDATE Run SET status='DONE', engineVersion=<git_sha>, exports_blob_urls=[...]`. COMMIT.
7. Desktop's poll sees `status='DONE'`, fetches presigned-GET URLs for layout + thumbnail, renders. Download buttons each fetch their own presigned URL on click.

Cancel flow: Desktop POST B30 (`cancelRunV2`, already shipped). mvp_api flips status + inserts refund + decrements entitlement in one tx (D16). In-flight Lambda's `FOR UPDATE` check sees CANCELLED at completion and aborts (D16).

Fail flow: Lambda's top-level try/except catches engine errors, runs the transactional refund pattern directly via psycopg2 (D17). DLQ catches Lambdas that crash before writing FAILED. Reconciler (D18) sweeps any remaining stuck-RUNNING.

5. Schema State

Already in place (no migrations required v1): - `Run` with status enum (RUNNING, DONE, CANCELLED, FAILED), `cancelledAt`, `failedAt`, `failureReason` (B29). - `UsageRecord` with `kind`, `count`, `refundsRecordId` self-relation (B29). - `Project.boundaryGeojson`, `Project.edits` (B3, B26). - Idempotency on `(userId, idempotencyKey)` (B2). - `Run.exportsBlobUrls Json @default("[]")` (B4) — slot for the artifact array populated by D12.

Adds in C7: - `Run.engineVersion: String?` — git SHA of the Lambda image that produced the result. - `Run.status` enum extended to include `QUEUED` (verify whether the existing string column accepts this without migration; new index entry).

Future-ladder (not built): - `Slice` table (per-plot fan-out, v3 only, §10).

6. Wire Contracts

Surface	Method	Path	Trigger	Notes
Project create	POST	`/v2/projects` (B11)	desktop / mobile	unchanged
KMZ upload URL	POST	`/v2/blobs/kmz-upload-url` (B6)	desktop / mobile	unchanged
Project KMZ parse	POST	`/v2/projects/:id/parse-kmz`	desktop / mobile	**NEW** (C4) — sync invokes parse-kmz Lambda
Run create + dispatch	POST	`/v2/projects/:id/runs` (B16)	desktop / mobile	**AMENDED** (C7) — adds SQS publish
Run poll	GET	`/v2/projects/:id/runs/:runId` (B17)	desktop / mobile	unchanged shape; carries new status states
Run cancel	POST	`/v2/projects/:id/runs/:runId/cancel` (B30)	desktop / mobile	already shipped; desktop wires in C10
Run delete	DELETE			unchanged

Surface	Method	Path	Trigger	Notes
		`/v2/projects/:id/runs/:runId` (B18)	desktop / mobile	
Export download URL	GET	`/v2/projects/:id/runs/:runId/exports/:type`	desktop / mobile	**NEW** (C17) — mints presigned-GET; `type ∈ {dxf, pdf, kmz}`
Water-detect dispatch	POST	`/v2/projects/:id/water-detect`	desktop / mobile	**NEW** (C16) — publishes to SQS
Stuck-run reconciler	(cron)	internal	Vercel Cron / equivalent	**NEW** (C13)

All routes use license-key bearer auth (`Authorization: Bearer sl_live_*`) via the existing `licenseKeyAuth` middleware. No new auth scheme.

7. Repo Layout (target end state)

```
pv_layout_project/
├── apps/
│   ├── desktop/                          (Tauri + React;
thinner)
│   ├── mvp_api/                          (Hono/Bun + new SQS
lib + new Lambda invoke)
│   ├── mvp_web/
│   └── mvp_admin/
├── packages/
│   ├── entitlements-client/              (extended in C9, C17)
│   ├── mvp_db/                           (one migration in C7
for engineVersion)
│   ├── shared/
│   ├── sidecar-client/                   (DELETED in C19)
│   └── ui/, ui-desktop/, ...
└── python/
    ├── pvlayout_core/                    ← extracted in C2; the
engineering asset
    │   ├── pyproject.toml
    │   ├── pvlayout_core/
    │   └── tests/
    ├── pvlayout_engine/                  ← DELETED in C19
    └── lambdas/                          ← new in C3
        ├── parse-kmz/
        │   ├── pyproject.toml
        │   ├── parse_kmz_lambda/
        │   ├── Dockerfile
        │   └── tests/
```

```
├── compute-layout/
│   ├── pyproject.toml
│   ├── compute_layout_lambda/
│   │   ├── handler.py
│   │   └── db.py
│   ├── Dockerfile
│   └── tests/
└── detect-water/
    └── (same shape)
```

`pvlayout_core` consumed by Lambdas via `[tool.uv.sources]` path dep locally; via Dockerfile `COPY` at image build (D5).

8. Migration Sequencing

Sidecar coexists with Lambdas during the cutover so we never have both desktops broken at once. Per route, the desktop has a feature flag (`USE_CLOUD_PARSE`, `USE_CLOUD_LAYOUT`, etc.) that toggles between sidecar and cloud paths. We flip flags one at a time after each Lambda lands and is verified.

Order: 1. Doc cleanup (C1) — clear contradictions before writing new code. 2. `pvlayout_core` extracted (C2) — domain code is now independently versionable. 3. Lambda monorepo scaffolding + first Lambda (C3, C4) — `parse-kmz` cuts over via flag. 4. SQS + heavy Lambda (C5, C6, C7, C8) — `compute-layout` builds and tests in isolation, then wires. 5. Desktop migration off sidecar (C9) — flag flip; sidecar still runs but is unused for layout. 6. Cancel + fail wiring (C10, C11, C12, C13). 7. Visibility (C14, C15). 8. Remaining workloads (C16, C17). 9. Energy-yield wiring (C18) — brainstorm-first per row spec; cloud Lambda branch + Inspector tab. 10. Sidecar deletion (C19) — when every flag is permanently flipped. 11. Mobile parity verification (C20) — no new code; confirm forms. 12. Production cutover signoff (C21).

9. Implementation Rows

Each row is one cold-session unit. Status is one of `todo | in-progress | done | blocked`. Rows reference locked-decision IDs (D1–D23) to constrain re-debate. Child sessions invoke `superpowers:writing-plans` per row, then `superpowers:executing-plans` (TDD).

Row template fields: `Status`, `Depends`, `Tier`, `Brainstorm-first` (only on rows with open design or UX questions; absence = writing-plans-direct OK; governed by §11.3), `Goal`, `Locked`, `Open verifications`, `Acceptance`, `Smoke trigger` (only on rows where customer behavior changes or new infrastructure goes live; absence = no smoke required; governed by §11.2), `Out of scope`.

Phase A — Foundation

C1 — Doc cleanup

```
Status:   done (2026-05-03)
Depends:  none
Tier:     T1

Goal
  Delete or annotate every contradictory / superseded doc so the
  codebase has one source of truth for cloud-offload (this spec).
  See §14 for the full disposition table.

Locked
  D1, D2, D9 (the halted B32 plan is wrong; delete it)

Open verifications
  - Read each doc in the §14 kill list before acting.
  - Confirm referenced-from links (e.g., post-parity-v2-backend-
plan.md
    rows B27-B34) are updated to point at this spec.

Acceptance
  - Halted plan file deleted (`docs/superpowers/plans/2026-05-02-
b32-failed-runs-path.md`). ✅
  - Cable PRD + offload-feasibility memo annotated SUPERSEDED with
link
    to this spec in their headers. ✅
  - B27 memo §B.6 amended (the "internal endpoint sidecar
callback"
    paragraph rewritten to point at D9 + D17). ✅
  - post-parity-v2-backend-plan.md B28-B34 row group amended to
    redirect at this spec's C-rows (B28 → C14; B29/B30 done
historical;
    B31 → C11; B32 → C12; B33 → C10; B34 → C15). ✅
  - docs/PLAN.md "renewable_energy session" + Spike 2 references
    amended to reflect post-merge reality + cloud-offload spec. ✅
  - Spec §14 path typo fixed (was `docs/post-parity/PLAN.md`, now
    `docs/PLAN.md`). ✅
  - Atomic commit: `docs(c1): cloud-offload spec lockin + kill
stale docs`.

Out of scope
  - Code changes.
  - Deleting historical findings (`docs/post-parity/findings/*`
stays).

Shipped: 2026-05-03 — executed inline in the brainstorming session
that produced the master spec (§9 row table); commit ref appended
at
PR/push time. No separate writing-plans output (row was small and
mechanical; §14 kill list was the plan).
```

C2 — Extract `pvlayout_core` to standalone

```
Status:   done (2026-05-03)
Depends:  C1
Tier:     T2

Goal
  Move `python/pvlayout_engine/pvlayout_core/` to
  `python/pvlayout_core/` with its own pyproject.toml, tests, and
CI
  gate. The engine layer (`pvlayout_engine/`) keeps depending on
it as
  a path dep so the existing sidecar continues to work during
cutover.

Locked
  D4, D6

Open verifications
  - Inventory all `from pvlayout_core...` imports across the repo;
    none should break post-move.
  - Verify pytest + uv configs work standalone in the new
location.
  - Verify the existing sidecar tests still pass after re-pointing
    its dep path.

Acceptance
  - `python/pvlayout_core/pyproject.toml` exists; `uv sync` clean.
  - All existing tests under `pvlayout_core/tests/` pass under
    `cd python/pvlayout_core && uv run pytest`.
  - `python/pvlayout_engine/pyproject.toml` updated to reference
    `pvlayout_core` as `{path = "../pvlayout_core", editable =
true}`.
  - All sidecar tests still pass.
  - CI gate added: `python/pvlayout_core/` has its own pytest job.

Out of scope
  - Code changes inside pvlayout_core (move only).
  - Lambda use of pvlayout_core (that's C3+).

Plan:     docs/superpowers/plans/2026-05-03-c2-extract-pvlayout-
core.md
Shipped:  2026-05-03 — pvlayout_core extracted to standalone uv
          package (own pyproject + tests + CI gate). Engine
consumes
          via editable path-dep in [tool.uv.sources]. 39
pvlayout_core
          tests pass standalone; 89 engine tests still pass (post-
move
          baseline 128 = 39 + 89). Plan-text gap caught at smoke:
          5 additional engine integration tests used a different
KMZ
          path pattern not enumerated in §Task 7; fixed inline at
```

```
the
          Task 9 verification gate (commit 502b6c3).
```

Phase B — First Lambda end-to-end (proves the pattern)

C3 — Lambda monorepo scaffolding

```
Status:   todo
Depends:  C2
Tier:     T2

Goal
  Establish `python/lambdas/` with the conventions, shared
Dockerfile
  template, CI matrix workflow, and ECR repo naming. The first
  Lambda (parse-kmz, C4) lands using this scaffolding.

Locked
  D5, D10, D22

Open verifications
  - Confirm AWS account `378240665051` + region `ap-south-1` is
the
    target (per CLAUDE.md §4.2).
  - Verify GitHub Actions has OIDC role for ECR push; check
existing
    workflows for the pattern (likely none post-merge — needs
setup).
  - Decide ECR repo prefix (`pvlayout/` recommended; document
choice).

Acceptance
  - `python/lambdas/README.md` documents the convention (one
folder
    per Lambda, pyproject.toml + Dockerfile + handler/ + tests/).
  - Shared Dockerfile template documented or symlinked.
  - `.github/workflows/build-lambdas.yml` matrix workflow
scaffolded;
    initially empty matrix or with parse-kmz only.
  - Naming convention recorded:
      Lambda fn:  pvlayout-<purpose>-<env>
      ECR repo:   pvlayout/<purpose>
      SQS queue:  pvlayout-<purpose>-jobs + -dlq

Out of scope
  - Any Lambda code (that's C4).
  - IaC.
```

C4 — `parse-kmz` Lambda end-to-end

```
Status:           todo
Depends:          C3
```

```
Tier:             T2 (build + integration test against real Lambda
+ mvp_api)
Brainstorm-first: yes — first Lambda; pattern decisions (IAM,
invoke timeout config, error response shape, exact wire contract
vs sidecar) cascade to the 4 downstream Lambdas. Get them right
once.

Goal
  Ship the first cloud Lambda. Replaces sidecar /parse-kmz behind
a
  feature flag. Sync HTTP-invoked from mvp_api. Validates the
entire
  monorepo Lambda mechanism on the simplest workload before
scaling
  to compute-layout.

Locked
  D2, D5, D7, D10

Open verifications
  - Confirm C2 is done (pvlayout_core extracted standalone).
  - Verify Vercel function timeout for apps/mvp_api accommodates
~2s
    sync wait + Lambda cold start (~3-5s first call); see §11.
  - Verify @aws-sdk/client-lambda is NOT already in apps/mvp_api;
    install if missing.
  - Read current sidecar parse-kmz handler in pvlayout_engine/
routes/
    to capture exact request/response contract; mirror it.

Acceptance
  - python/lambdas/parse-kmz/ exists with full structure.
  - Lambda pvlayout-parse-kmz-staging deployed and invokeable.
  - New mvp_api route POST /v2/projects/:id/parse-kmz invokes
Lambda
    sync, returns parsed boundaries in V2 envelope.
  - Desktop's useCreateProject flow swaps from sidecar.parseKmz to
    mvp_api parseKmzV2 behind feature flag USE_CLOUD_PARSE
(default
    off).
  - Integration test: real KMZ → mvp_api → Lambda → parsed JSON.
  - Sidecar's /parse-kmz STAYS alive (deletion is C19).

Smoke trigger (Arun, per sec 11.2)
  Local:    Tauri pointed at localhost:3003 mvp_api with
            USE_CLOUD_PARSE=true; pick a real customer KMZ;
observe
            "Reading boundaries..." spinner; verify parsed
boundaries
            render on canvas matching legacy sidecar parse.
  Staging:  AWS-only smoke against staging Lambda. Invoke
            pvlayout-parse-kmz-staging directly with a real KMZ
            payload via `aws lambda invoke --function-name
            pvlayout-parse-kmz-staging --payload <b64-kmz> ...`;
```

```
            verify CloudWatch shows the invocation; verify
response
            shape matches the V2 envelope and the parsed
boundaries
            count matches the legacy sidecar output. (No staging
            mvp_api exists; the mvp_api ↔ Lambda wire is verified
            at Prod.)
  Prod:     deferred to phase-end (C21).

Out of scope
  - Production deployment (staging only at this row).
  - Lambda timeout tuning beyond defaults.
  - IaC.
  - Cache layer.
```

Phase C — Heavy compute Lambda + orchestrator changes

C5 — SQS infrastructure for compute-layout

```
Status:   todo
Depends:  C3
Tier:     T1

Goal
  Provision the SQS queue + DLQ + IAM for compute-layout in
staging
  + prod, manually via aws CLI. Document the provisioning steps
for
  the eventual IaC migration.

Locked
  D8

Open verifications
  - Verify IAM user `renewable-energy-app` (CLAUDE.md §4.2) has
    SQS permissions; extend the inline policy if not.
  - Verify VPC requirements for the Lambda → RDS connection
    (probably needs VPC config to reach RDS; check existing
infra).

Acceptance
  - SQS queues: pvlayout-compute-layout-jobs-{staging,prod} +
    pvlayout-compute-layout-jobs-dlq-{staging,prod}.
  - RedrivePolicy: maxReceiveCount = 3; visibility timeout = 1020s
    (Lambda 14min × 1.2 + buffer).
  - CloudWatch alarm on DLQ ApproximateNumberOfMessages > 0 for
>5min.
  - `docs/AWS_RESOURCES.md` updated with the new queue ARNs +
    provisioning commands.
  - IAM policy updated additively; verified by aws sts.

Out of scope
```

```
  - Any Lambda code.
  - IaC.
```

C6 — `compute-layout` Lambda (handler + Dockerfile + tests, no SQS trigger yet)

```
Status:   todo
Depends:  C2, C3
Tier:     T2 (golden-test gate against pvlayout_core)

Goal
  Build the compute-layout Lambda image with the all-in-one
handler:
  fetches KMZ from S3, runs pvlayout_core.run_layout, renders DXF
+
  PDF + KMZ + thumbnail inline, PUTs all artifacts to S3, writes
  status/engineVersion via psycopg2. Initially testable in
isolation
  (no SQS); tests invoke the handler directly with mock event.

  Cable routing (AC + DC) is a sub-step of
pvlayout_core.run_layout
  controlled by params.enable_cable_calc; conditional inclusion in
  DXF/PDF/KMZ exports is already handled by the existing exporter
  modules. The Lambda doesn't separately decide — it trusts what's
  in params (D23). Layer-2 cable feature gating moves to mvp_api
in
  C7.

Locked
  D5, D9, D10, D11, D12, D13, D14, D17, D20, D21, D23

Open verifications
  - Confirm the existing sidecar /export-{dxf,pdf,kmz} routes use
the
    same pvlayout_core exporters we'll call from Lambda (no
    sidecar-only logic in the export path).
  - Confirm thumbnail rendering uses the existing pvlayout_core
    thumbnail module per memo v3.
  - Verify the Run schema columns (status, engineVersion to be
    added in C7, exportsBlobUrls) match the Lambda's UPDATE
    statement.
  - Test golden-equivalence: same KMZ → same layout JSON whether
    run via sidecar today or via Lambda image (cross-engine
    consistency, modulo engineVersion).

Acceptance
  - python/lambdas/compute-layout/ full structure with Dockerfile
    that COPYs pvlayout_core.
  - Image builds locally via docker build.
  - Handler implements the full lifecycle:
      START → SELECT FOR UPDATE → if CANCELLED abort
                                → if QUEUED flip to RUNNING
```

```
                                  → fetch KMZ from S3
                                  → run_layout
                                  → render dxf, pdf, kmz, thumbnail
                                    (each in its own try/except)
                                  → PUT all artifacts to S3
                                  → SELECT FOR UPDATE again (D16)
                                  → if CANCELLED abort + S3 cleanup
                                  → else UPDATE status=DONE,
                                         engineVersion=git_sha,
                                         exports_blob_urls=[...]
      EXCEPTION → UPDATE status=FAILED + INSERT refund row (D17).
  - Unit tests cover all branches with mocked S3 + DB.
  - Golden test: against complex-plant-layout.kmz fixture, output
is
    byte-equal (modulo engineVersion field) to current sidecar
output.

Out of scope
  - SQS trigger wiring (C8).
  - mvp_api orchestrator changes (C7).
  - Production deploy.
```

C7 — mvp_api orchestrator: SQS publish + engineVersion column

```
Status:           todo
Depends:          C5, C6
Tier:             T2 (integration test mocks SQS)
Brainstorm-first: yes — multi-feature gating semantics (D23) +
publish-then-commit tx-and-SQS interaction with B16's existing
idempotency machinery. Subtle ordering questions; non-obvious
failure modes.

Goal
  Extend B16 (createRunV2) to publish to SQS after Run.create
within
  the same transaction (publish-then-commit, D19). Add migration
for
  Run.engineVersion. Add Run.status='QUEUED' as a valid initial
  state. **Move layer-2 cable feature gating from sidecar to B16
  (per D23):** today the sidecar's
require_feature("cable_routing")
  enforces; once C19 deletes the sidecar, B16 must already enforce
  this or cable gating breaks. Energy gating (when C18 lands)
joins
  the same pattern.

Locked
  D14, D15, D19, D21, D23

Open verifications
  - Verify Run.status enum's current values; if string column
without
    enum constraint, no DDL needed for QUEUED — confirm in schema.
```

- Verify @aws-sdk/client-sqs is NOT yet a mvp_api dep; install.
- Verify the existing reportUsage transaction in B16 — extend it, don't replace.
- Verify Vercel function timeout includes margin for SQS SendMessage (~50-100ms typical, fine).
- **Read the current sidecar's gating chain to inventory which feature keys gate which params.** Specifically: `require_feature("cable_routing")` on the layout route guarded by `params.enable_cable_calc`. Capture the exact mapping so B16's multi-feature check is faithful.
- Verify B16's existing `findFeaturePool` + entitlement-debit path can be extended to a multi-feature check without breaking the idempotency contract.

Acceptance
- Migration adds Run.engineVersion: String? column.
- apps/mvp_api/src/lib/sqs.ts publishes to compute-layout queue.
- B16 wraps existing tx + SQS publish; rollback on SQS error.
- **B16 enforces multi-feature gating per D23**: if `params.enable_cable_calc=true`, user must have `cable_routing` in availableFeatures; reject 402 if missing. Future-proofed for energy keys when C18 lands. Today's cable-via-sidecar gating remains live until C19 deletes the sidecar.
- Integration test: mock SQS → POST /v2/projects/:id/runs creates Run with status=QUEUED + publishes 1 message; if SQS throws, no Run row created.
- Integration test: cable-flag=true + user lacks cable_routing → 402, no Run row created.
- Run.status='QUEUED' returned in the response on success.

Smoke trigger (Arun, per sec 11.2)
Local: curl POST /v2/projects/:id/runs with cable_calc=true on a Free-tier license key → expect 402 with feature_key citing cable_routing. Then same on Pro key → expect 201 + Run.status=QUEUED. Verify SQS message in dev queue.
Staging: DB migration smoke — apply the engineVersion migration against staging RDS first to verify it lands cleanly: `set -a; . ./.env.staging; set +a; bunx prisma migrate deploy --schema=packages/mvp_db/prisma/schema.prisma`. Verify with `bunx prisma migrate status ...`. (CD applies the same migration against prod RDS on PR merge.) No app-code smoke at this level — staging

```
            mvp_api does not exist.
  Prod:     repeat the two 402 / 201 cases against
            api.solarlayout.in with Arun's Free-tier and Pro-tier
            prod license keys. Verify the prod compute-layout SQS
            queue receives the message on the 201 path; confirm no
            message published on the 402 path (rollback worked).
            Verify a fresh Run row carries the new engineVersion
            column (NULL until C8 lands; non-NULL afterward).

Out of scope
  - Lambda actually consuming the message (C8).
  - Outbox pattern.
  - Energy gating (lands with C18 — same pattern, additive).
```

C8 — Wire `compute-layout` Lambda to SQS trigger (end-to-end)

```
Status:           todo
Depends:          C5, C6, C7
Tier:             T2 (integration: real SQS + real Lambda + real
RDS)
Brainstorm-first: yes — first full async cloud end-to-end. IAM
execution role scope, VPC config for RDS reach, SQS batch-size
choice, visibility-timeout sizing, DLQ alarm thresholds — design
decisions that establish the pattern for all subsequent SQS
Lambdas.

Goal
  Connect the compute-layout Lambda to the SQS queue as event
source.
  End-to-end: mvp_api creates Run + publishes → Lambda picks up →
  writes RDS + S3 → desktop polling sees DONE. Staging only.

Locked
  D8, D9, D14

Open verifications
  - Confirm Lambda's IAM execution role has sqs:ReceiveMessage,
    sqs:DeleteMessage on the queue + s3:* on the projects bucket
    + RDS connectivity (VPC).
  - Verify visibility timeout is correct given Lambda timeout.
  - Verify max batch size = 1 (each Run is one message; we don't
want
    Lambda batching multiple Runs into one invocation).

Acceptance
  - Lambda function pvlayout-compute-layout-staging deployed with
SQS
    event source mapping.
  - End-to-end staging test: trigger via Tauri (or curl), observe
    Run.status transitions QUEUED → RUNNING → DONE, observe S3
    artifacts.
  - DLQ stays empty under normal traffic.
```

```
Smoke trigger (Arun, per sec 11.2) — first full async cloud smoke
  Local:    LocalStack (or skip; staging AWS is the real test).
  Staging:  AWS-resource end-to-end against staging infra. Hand-
            craft an SQS message body (run id + project id +
params)
            and `aws sqs send-message --queue-url <staging-
compute-
            layout-jobs> --message-body ...`; observe pvlayout-
            compute-layout-staging Lambda fire (CloudWatch);
verify
            staging RDS Run row transitions QUEUED → RUNNING →
DONE;
            verify all 5 S3 artifacts (layout.json + 3 exports +
            thumbnail) land in solarlayout-staging-projects
bucket;
            DLQ empty. Then a complex-plant payload (~4min):
confirm
            Lambda doesn't time out; final status DONE. (No
staging
            mvp_api exists — Tauri-triggered end-to-end at Prod.)
  Prod:     deferred to phase-end (C9 cutover smoke).

Out of scope
  - Production deploy.
  - Cancel-marker check (C11) — Lambda just runs to DONE in this
row.
  - Fail handling integration (C12).
  - Desktop UI changes (C9).
```

C9 — Desktop migration off sidecar `/layout/jobs`

```
Status:           todo
Depends:          C8
Tier:             T2 (live verification on staging fixtures)
Brainstorm-first: yes — first user-facing cutover. Multiple state-
shape decisions (LayoutJobState reduction, RunningPin redesign),
polling cadence, feature-flag granularity, what to do on cloud-
flag-off fallback. Read current useGenerateLayout end-to-end
before deciding.

Goal
  Switch useGenerateLayout from sidecar /layout/jobs polling to
  mvp_api Run.status polling. Behind feature flag
USE_CLOUD_LAYOUT.
  Remove per-plot progress UI from RunningPin (D11 implication).

Locked
  D1, D11

Open verifications
  - Read current useGenerateLayout.ts and inventory every
reference
    to sidecarClient.startLayoutJob/getLayoutJob/cancelLayoutJob.
```

```
  - Identify what RunningPin reads from useCurrentLayoutJobStore
    (LayoutJobState.plots[]) and what its replacement looks like
    (a single status + elapsed timer).
  - Verify B17's response carries status field (post-B30: yes).
  - Verify polling cadence ~2s is fine for mvp_api (no rate-limit
    concern at single-user volume).

Acceptance
  - useGenerateLayout: when USE_CLOUD_LAYOUT, skips sidecar
entirely;
    polls mvp_api getRunV2 every 2s until DONE/FAILED/CANCELLED.
  - LayoutJobState reduced to {status, elapsed_estimated_pct?}.
  - RunningPin renders simplified progress (single bar or
spinner).
  - All existing tests pass; new tests cover the cloud-flag
branch.
  - Live: staging end-to-end verified.

Smoke trigger (Arun, per sec 11.2) — first user-facing cutover
smoke
  Local:    Tauri with USE_CLOUD_LAYOUT=true, sidecar still
running
            but unused for layout; click Generate; verify spinner
+
            polling against mvp_api dev server; canvas renders
            cloud-produced layout JSON; per-plot UI absent.
  Prod:     PRODUCTION CUTOVER for layout — flip USE_CLOUD_LAYOUT
in
            a release build. Run the full Generate flow on
            phaseboundary2 + complex-plant fixtures via Arun's
prod
            license against api.solarlayout.in; verify polling
            cadence and canvas hydration match parity with
sidecar-
            rendered output (modulo engineVersion); capture wall-
            clock vs sidecar baseline.
```

Phase D — Cancel + Fail semantics

C10 — Desktop cancel modal + cancelRunV2 wiring (was B33)

```
Status:           todo
Depends:          C9
Tier:             T2
Brainstorm-first: yes — modal copy + UX flow + state-machine
decisions (what does the polling loop see when cancel succeeds vs
when cancel races completion). Non-obvious because of
LayoutJobCancelledError + the parity-era code paths being
replaced.

Goal
  When user clicks Cancel: show confirmation modal (B27 §B.7),
call
```

```
  cancelRunV2 (B30, already shipped), let polling loop see
CANCELLED.
  Replace the parity-era best-effort deleteRunV2 cleanup.

Locked
  D3 (no incremental edit endpoints), D16

Open verifications
  - Verify B30 endpoint shape from runs.service.ts:cancelRun.
  - Verify entitlements-client doesn't yet have cancelRunV2
method;
    add per the same shape as deleteRunV2.
  - Verify the existing LayoutJobCancelledError handling in
    useGenerateLayout still applies (it should).

Acceptance
  - entitlements-client.cancelRunV2(projectId, runId) added with
V2
    envelope handling.
  - Confirmation modal: "Cancel this generation? You'll lose work
    in progress. Your calculation will be refunded." Two buttons.
  - On confirm → cancelRunV2 → polling loop sees CANCELLED →
throws
    LayoutJobCancelledError as before.
  - The pre-existing deleteRunV2 cleanup is removed.
  - Tests cover modal + cancel + refund visible in subsequent
    /v2/entitlements call.

Smoke trigger (Arun, per sec 11.2) — money flow
  Local:    Tauri click Generate; mid-flight click Cancel; observe
            modal copy; confirm cancel; verify entitlement
            remainingCalculations restored (refund); verify Run
            row shows status=CANCELLED with deletedAt=null.
  Prod:     same flow on Arun's prod license against
            api.solarlayout.in. Verify the
UsageRecord(kind='refund')
            row exists in prod RDS with correct refundsRecordId.
            Don't trigger more cancels than needed — each consumes
            a real entitlement cycle on the live Subscription
chain.

Out of scope
  - Lambda-side cancel-marker check (C11).
  - RunsList rendering of CANCELLED state (C14).
```

C11 — Lambda cancel-marker check

```
Status:           todo
Depends:          C10
Tier:             T2 (race-test deterministically)
Brainstorm-first: yes — race semantics + S3 cleanup edge cases
(which artifacts already PUT? best-effort vs strict cleanup? DLQ
implications if cleanup itself fails?). Subtle correctness
```

territory.

Goal
  In compute-layout Lambda, before flipping Run.status to DONE, do
  SELECT...FOR UPDATE on Run.status. If CANCELLED, abort the
upload
  and best-effort delete any S3 objects already PUT. Implements
  D16's "cancel always wins until DONE committed."

Locked
  D16

Open verifications
  - Confirm C6's handler structure already has the post-compute
    transactional block; this row adds the FOR UPDATE re-read.
  - Verify S3 DeleteObject IAM permission on Lambda role.

Acceptance
  - Handler does FOR UPDATE re-read; on CANCELLED, S3 cleanup +
    return without flipping status (status stays CANCELLED from
B30).
  - Race test: parallel run that issues B30 cancel mid-handler;
    final state is CANCELLED + no DONE update + S3 best-effort
    cleanup.
  - Integration test confirms refund row exists exactly once
    (B30's, not duplicated by Lambda).

Smoke trigger (Arun, per sec 11.2) — race-condition smoke
  Local: skip — race needs real timing; covered at Prod.
  Prod: Tauri click Generate on a small fixture
(phaseboundary2,
            ~4s) against api.solarlayout.in; race the Cancel click
            to land DURING the Lambda's compute window. Verify
            (a) Run ends CANCELLED; (b) exactly ONE refund row in
            prod RDS; (c) S3 layout/exports either absent OR
            explicitly cleaned up by Lambda; (d) no "DONE →
            CANCELLED → DONE" status flap. Run minimum number of
            attempts to land the race window — each costs a real
            calc on the live Subscription chain.

Out of scope
  - Mid-run cancel polling (Lambda checks ONLY at completion, per
    D16 — keeps cost low).

C12 — Lambda fail path

Status: todo
Depends: C8
Tier: T2
Brainstorm-first: yes — error-context taxonomy (failureReason
format, stack-trace truncation strategy), partial-progress S3
artifact cleanup behavior, distinguishing system errors from user-
input errors at v1 (B27 §A.1 says one badge — but failureReason

```
text matters for support).

Goal
  Top-level try/except in compute-layout handler runs the
  transactional FAILED + refund pattern via psycopg2 (D17).
Mirrors
  B30's cancel transaction.

Locked
  D9, D17

Open verifications
  - Read B30 cancelRun service to mirror its tx pattern in Python.
  - Verify failureReason column accepts up to 500 chars (defensive
    cap on stack-trace text).
  - Confirm Lambda's role can write UsageRecord rows.

Acceptance
  - Handler-level try/except wraps everything; on exception:
      BEGIN
      SELECT Run, UsageRecord (charge) FOR UPDATE
      UPDATE Run SET status='FAILED', failedAt=NOW(),
                     failureReason=<stringified>
      INSERT UsageRecord (count=-1, kind='refund',
                          refundsRecordId=<charge.id>)
      UPDATE Entitlement SET usedCalculations -= 1
      COMMIT
  - Tests inject a fault (e.g., mock pvlayout_core.run_layout to
    throw); verify status flip, refund row, entitlement decrement.
  - Idempotent on redelivery: WHERE status IN ('RUNNING') guards.

Smoke trigger (Arun, per sec 11.2) — money flow on system error
  Local:    Inject a fault into pvlayout_core (e.g., set an
            explicit raise mid-handler in dev); trigger Generate;
            verify Run.status=FAILED + refund row + entitlement
            decremented + failureReason populated.
  Prod:     use a deliberately-broken KMZ that triggers a known
            engine error (Prasanta or test-fixture sourced)
against
            api.solarlayout.in on Arun's prod license; same
            observations against prod RDS. DLQ should NOT receive
            (Lambda caught + handled the failure cleanly).

Out of scope
  - Failure-type taxonomy (one badge per B27 §A.1).
  - Stuck-RUNNING reconciler (C13).
```

C13 — Stuck-RUNNING reconciler

```
Status:           todo
Depends:          C12
Tier:             T1
Brainstorm-first: yes — host-platform decision (Vercel Cron vs
```

admin endpoint vs standalone scheduler), threshold value, observability shape, behavior when reconciler-itself fails mid-sweep.

Goal
  Scheduled job sweeps Runs in RUNNING state older than N minutes (default 30) and flips them to FAILED with reason 'reconciler:timeout' + refund row. Catches Lambdas that crash before writing FAILED.

Locked
  D18

Open verifications
  - Decide host: Vercel Cron, mvp_api manual admin trigger, or standalone scheduler. Vercel Cron preferred; verify availability on the project's plan.
  - Decide N (default 30 min; configurable via env).
  - Verify the cron handler can run the same transactional pattern as C12 (call shared service function).

Acceptance
  - apps/mvp_api/src/modules/runs/reconciler.ts shared function.
  - Cron route (or admin route) wraps the reconciler.
  - Test: insert a Run with status=RUNNING + createdAt 60min ago → invoke reconciler → status flipped, refund written.
  - Logging: counts of swept Runs per invocation.

Smoke trigger (Arun, per sec 11.2) — backstop for crashed Lambdas
  Local: insert a Run row directly in dev RDS with status=RUNNING
            + createdAt = NOW()-1h; trigger reconciler manually; verify FAILED + refund.
  Staging: same Run-row insertion against staging RDS via psql to confirm the SQL behaves correctly against a real RDS instance + indexes. (No Vercel staging app exists, so the cron-trigger half cannot run here — that runs at Prod.)
  Prod: verify the Vercel Cron is wired in prod (Vercel dashboard → mvp_api → Cron); insert one stuck Run row in prod RDS via psql with createdAt 60+min in the past;
            wait for the next cron tick (or trigger via the admin-endpoint fallback if implemented); verify Run.status flips to FAILED, a refund UsageRecord row lands, and the cron log records the swept count. Clean up the test row afterward (or leave it — the FAILED status + refund pair is harmless).

Out of scope
  - Configurable per-tier timeout (one global value v1).
  - Auto-retry on stuck Run (just FAILED + refund; user retries).

Phase E — Visibility + history

C14 — Visible cancelled / failed runs in RunsList + RecentsView (was B28)

```
Status:           todo
Depends:          C10
Tier:             T2
Brainstorm-first: yes — UI variant decisions (icon set, badge
copy, fallback layouts), interaction contract (clickable?
selectable? tooltip on the failureReason?), RecentsView thumbnail-
fallback chain.

Goal
  RunsList renders status-aware cards: DONE (existing card),
  CANCELLED (Ban icon + "Cancelled" badge), FAILED (AlertCircle +
  "Layout failed"), QUEUED/RUNNING (spinner). RecentsView's
  project-card honors status for thumbnail fallback.

Locked
  D15

Open verifications
  - Confirm RunSummary wire shape carries status (post-B30:
extended
    on RunWire; verify it's also on RunSummary used by B12/B15).
  - Read RunsList component to find the existing card variants.
  - Read RecentsView's thumbnail fallback chain.

Acceptance
  - RunsList renders 4 status variants distinctly.
  - Cancelled/failed runs are NOT selectable (no canvas
hydration).
  - RecentsView project-card respects last-run status for
thumbnail
    placeholder.
  - Tests cover all status branches.

Smoke trigger (Arun, per sec 11.2) — visibility regression
detection
  Local:    seed dev RDS with one Run per status (DONE / CANCELLED
            / FAILED / RUNNING); open Tauri RunsList +
RecentsView;
            visually confirm 4 distinct variants render correctly.
  Prod:     verify against prod RDS state — Arun's prod account
            should already have at least one each of DONE /
            CANCELLED / FAILED Runs from prior C10 / C11 / C12
            smokes; if not, exercise once each. Open Tauri pointed
            at api.solarlayout.in; visually confirm 4 distinct
            variants render correctly; cancelled / failed cards
            are visually distinguishable from DONE; click does NOT
            hydrate canvas; thumbnail fallback respects status.
```

```
Out of scope
  - Re-running a failed run (deferred).
```

C15 — `/dashboard/usage` status badges (was B34)

```
Status:   todo
Depends:  C12
Tier:     T1

Goal
  mvp_web's calc-history table joins Run for status, renders three
  badges (Completed / Cancelled / Failed). Refund rows hidden;
  charge rows show with status. Quota math reflects refunds via
  invisible math.

Locked
  D17

Open verifications
  - Read existing /dashboard/usage page in apps/mvp_web/.
  - Verify the listing endpoint filters or can be extended to
filter
    WHERE kind='charge'.
  - Verify Run-join wire shape.

Acceptance
  - Listing endpoint filters kind='charge' + joins Run.status.
  - mvp_web table renders 3 badges.
  - Refund rows invisible to customer; quota number shows
correctly.
  - Tests cover all three states.

Smoke trigger (Arun, per sec 11.2) — customer-visible smoke
  Local:    seed dev RDS with 3 UsageRecord rows (one each for
            DONE/CANCELLED/FAILED Run); load mvp_web /dashboard/
            usage in browser; verify 3 badges + correct quota
            remaining.
  Prod:     load solarlayout.in/dashboard/usage in browser as
            Arun's prod Clerk session. Verify 3 badges render for
            the existing prod UsageRecord rows; correct quota
            remaining; refund rows are NOT visible in the table.

Out of scope
  - Customer-visible refund history.
  - Notifications on Failed.
```

Phase F — Remaining workloads

C16 — `detect-water` Lambda

```
Status:           todo
Depends:          C5, C8 (pattern reuse)
```

```
Tier:             T2
Brainstorm-first: yes — endpoint shape + lifecycle relationship to
Run (separate detection-job entity? attached to Run? stateless
write-through to Project.edits?). Row body explicitly notes this
is undecided.

Goal
  Port sidecar /detect-water to a SQS-triggered Lambda. Reuses the
  Lambda monorepo + SQS + RDS-direct patterns from compute-layout.
  Writes water polygons to Project.edits.water_obstructions[].

Locked
  D8, D9, D14

Open verifications
  - Read current sidecar detect_water route + satellite tile
fetch.
  - Verify Project.edits schema accommodates water_obstructions
    array (already opaque Json, fine).
  - Decide trigger endpoint shape: POST /v2/projects/:id/water-
detect
    returns immediately with a "detection job started" status?
    Or wire into the Run lifecycle? Likely separate from Run.

Acceptance
  - python/lambdas/detect-water/ full structure.
  - SQS queue + DLQ provisioned.
  - mvp_api route POST /v2/projects/:id/water-detect publishes
msg.
  - Lambda fetches KMZ, runs satellite detection, writes polygons
    via psycopg2 to Project.edits.
  - Desktop swaps from sidecar to mvp_api behind flag.
  - Sidecar route stays alive (deletion is C19).

Smoke trigger (Arun, per sec 11.2) — second SQS pattern, lighter
  Local:    skip — needs real satellite tile fetch.
  Staging:  AWS-only smoke against staging detect-water Lambda.
            Hand-craft an SQS message (project id + KMZ S3 key)
and
            `aws sqs send-message` to the staging detect-water
            queue; observe Lambda fire (CloudWatch); verify
            Project.edits.water_obstructions populates in staging
            RDS within ~60s; verify polygons match legacy sidecar
            output for a known fixture. (Tauri-triggered end-to-
end
            at Prod.)
  Prod:     Tauri click "Detect water" against api.solarlayout.in
            on a known fixture with water bodies; observe
            Project.edits.water_obstructions populated in prod RDS
            within ~60s; canvas overlays water polygons matching
            legacy sidecar output.

Out of scope
  - Lifecycle table for detection jobs (write-through is
```

```
sufficient).
  - Caching detected water polygons.
```

C17 — Download endpoints + desktop export migration

```
Status:           todo
Depends:          C8
Tier:             T2
Brainstorm-first: yes — endpoint shape (presigned URL minted on
each request vs cached? response carries signed URL or 302
redirects?), error UX when artifact missing, expired-URL UX, file-
naming convention for the user's saved download.

Goal
  mvp_api routes that mint presigned-GET URLs for the pre-rendered
  artifacts (D12). Desktop export buttons swap from sidecar
  /export-* to mvp_api download endpoints behind feature flag.

Locked
  D12

Open verifications
  - Confirm Run.exportsBlobUrls is populated by C6's compute-
layout
    Lambda.
  - Verify the deterministic key path:
    `projects/<userId>/<projectId>/runs/<runId>/exports/
run.<type>`.
  - Read existing Tauri export buttons + their sidecar calls.

Acceptance
  - GET /v2/projects/:id/runs/:runId/exports/:type returns
    {downloadUrl, expiresAt, sha256?, size?} or 404.
  - Desktop export buttons (DXF/PDF/KMZ) call mvp_api endpoint
    behind USE_CLOUD_EXPORTS flag.
  - Browser/Tauri shell handles the actual download via the URL.
  - Tests cover happy path + missing artifact (404) + expired URL.

Smoke trigger (Arun, per sec 11.2) — exports cutover smoke
  Local:    Tauri with USE_CLOUD_EXPORTS=true; pick a DONE Run
from
            prod RDS (or seed dev RDS with a synthesized DONE Run
            pointing at known prod S3 keys); click each of
            DXF/PDF/KMZ download; verify file lands on disk; open
            each (AutoCAD/Acrobat/Earth) and confirm contents
match
            legacy sidecar export.
  Prod:     PRODUCTION CUTOVER for exports — flip
USE_CLOUD_EXPORTS
            in a release build. Pick a Run on a customer-realistic
            plant from prod RDS via Arun's prod license; download
            all three formats; spot-check each. Especially: PDF
```

```
            includes correct project metadata, DXF layers correct,
            KMZ opens in Google Earth.
```

C18 — Energy-yield wiring (cloud Lambda branch + Inspector tab)

```
Status:           todo
Depends:          C8 (compute-layout Lambda live), C9 (desktop
migration), C10 (cancel modal — pattern reuse)
Tier:             T2 (cross-runtime: Lambda branch + desktop UI +
integration test)
Brainstorm-first: yes — explicit; multiple practical UX questions
undecided at spec time (always-compute-or-toggle, billedFeatureKey
shape, missing-entitlement Inspector tab UX, layout-only-run
"Compute energy" CTA, network-I/O timeout impact). Listed in
detail in this row's Goal section.

Goal
  Wire energy yield end-to-end through the cloud architecture.
  Lambda branches on user-requested-energy flag, calls
  pvlayout_core's energy module (already parity-working against
  legacy PVlayout_Advance), PUTs result to
Run.energyResultBlobUrl.
  Desktop Inspector "Energy" tab reads + displays. Feature-gating
  respected per existing entitlement system.

  This row's child session MUST start with
superpowers:brainstorming
  before writing-plans, per the spec sec 13 protocol. Practical
  realities not nailed down at spec time:

  - Does Generate Layout ALWAYS compute energy if the user has the
    entitlement, or is energy a separate user toggle? (Likely
toggle
    — Pro Plus user may have both layout-only and layout+energy
    runs; affects calc billing.)
  - billedFeatureKey for an energy-included run: plant_layout +
    secondary energy_yield, or a single combined key? Memory says
    energy_yield is its own feature key in the entitlement model.
  - Inspector Energy tab UX when entitlement is missing: empty
    state? Upsell chip? Greyed tab? UX decision.
  - Inspector Energy tab UX when run was layout-only (no energy
    computed): "Compute energy" CTA that fires a new Run? Hidden?
  - PVGIS / NASA POWER fetch (~30s of network I/O) inside the same
    Lambda invocation as layout compute — verify no Vercel/Lambda
    timeout impact (sec 11 V1).

Locked
  D11 (per-Run unit), D12 (one Lambda invocation), D13 (rolled in
  v1, gated, separate UI tab), D9 (Lambda direct-RDS), D23
(compound
  feature gating; energy is the energy-side instance of the same
  pattern C7 establishes for cables).
```

```
Open verifications
  - Read pvlayout_core/energy_calculator.py for the existing
    module's public API + dep on requests + PVGIS / NASA POWER
    endpoints.
  - Read the existing Tauri Inspector tab structure (LayoutPanel +
    siblings) to understand where the Energy tab mounts. Verify
    whether any Energy-tab scaffolding already exists (parity-
era).
  - Read entitlements service for energy_yield +
generation_estimates
    feature key handling — B16's findFeaturePool already supports
    both keys; verify.
  - Verify Run.energyResultBlobUrl column exists post-B4 (yes per
    schema state sec 5).

Acceptance
  - Brainstorm output committed at docs/superpowers/specs/<date>-
    energy-yield-design.md before writing-plans starts.
  - Lambda compute-layout branches on energy flag; if requested
AND
    entitlement present, calls energy_calculator + PUTs energy
    result.
  - Desktop Inspector renders Energy tab; reads
    Run.energyResultBlobUrl if present; renders empty / upsell
    state otherwise.
  - Feature gating: B16 rejects energy-class Run create with 402
if
    user lacks entitlement (existing machinery — verify).
  - End-to-end test: Pro Plus user → Generate with energy flag →
    Run.energyResultBlobUrl populated → desktop renders.
  - Free / Basic / Pro user: Energy tab shows upsell or empty
state;
    no 402 cascade in console.

Smoke trigger (Arun + Prasanta, per sec 11.2) — new gated feature
  Local:    Tauri on Pro Plus license; Generate with energy
            requested on a small fixture; verify energy result
            populates Inspector Energy tab; numbers match legacy
            (Prasanta validates solar-domain correctness on at
            least one fixture). Then on Pro license: verify upsell
            state in Energy tab; no console errors.
  Staging:  AWS-only smoke against staging compute-layout Lambda
            with energy flag in the SQS payload. Verify external
            calls (PVGIS / NASA POWER) succeed from the staging
            Lambda VPC; verify energy result blob lands in staging
            S3 (solarlayout-staging-projects); wall-clock
acceptable
            (<2min total). (Tauri-triggered Pro-Plus end-to-end at
            Prod.)
  Prod:     PRODUCTION CUTOVER for energy — flip the energy
            feature flag in a release build; Arun runs Generate-
            with-energy on a real Pro Plus license against
            api.solarlayout.in; numbers verified.
```

```
Out of scope
  - Functional split (separate compute-energy Lambda) — v2 future.
  - PVGIS / NASA POWER caching across Runs — defer.
  - Energy-only re-runs against an existing layout (re-using
cached
    layout result) — defer; brainstorm whether this is v1 or v2.
```

Phase G — Closeout

C19 — Sidecar deletion

```
Status:           todo
Depends:          C4, C9, C16, C17, C18 (every flag must be
permanently flipped)
Tier:             T3 (decision memo + multi-file deletion)
Brainstorm-first: yes — irreversibility risk management.
Brainstorm forces inventory of stragglers (any USE_CLOUD_* flag
still off? any sidecar import in test fixtures? any CI gate still
pointing at pvlayout_engine/?) before deletion is committed. The
brainstorm output IS most of the decision memo.

Goal
  Delete python/pvlayout_engine/, packages/sidecar-client/,
  apps/desktop/src-tauri/src/sidecar.rs (the process management),
  Tauri sidecar config, the USE_CLOUD_* feature flags, and every
  remaining sidecar reference. The end-state code reflects D2.

Locked
  D2

Open verifications
  - Verify every USE_CLOUD_* feature flag has been on (true) in
    production for at least 1 release cycle without rollback.
  - Inventory every remaining import of @solarlayout/sidecar-
client
    or sidecar functions; expect zero in production code paths.
  - Confirm Tauri build still works without the sidecar bundling.
  - Confirm CI no longer needs the sidecar pytest gate (it does —
    keep python/pvlayout_core/ tests; drop python/pvlayout_engine/
    tests).

Acceptance
  - python/pvlayout_engine/ deleted.
  - packages/sidecar-client/ deleted.
  - Tauri Rust sidecar process management removed.
  - tauri.conf.json sidecar entries removed.
  - All feature flags removed; cloud paths are the only paths.
  - CI gates updated; pre-commit gate from CLAUDE.md §8 still
green.
  - Decision memo at docs/initiatives/findings/YYYY-MM-DD-NNN-
    sidecar-deletion.md documenting the switchover dates and
    rollback context.
  - Atomic commit: `chore: delete sidecar; cloud paths are
```

canonical`.

Smoke trigger (Arun, per sec 11.2) — irreversibility smoke; FULL FUNCTIONAL REGRESSION

Local: fresh Tauri build off the deletion branch; full customer-flow smoke: license-key login → new project (parse-kmz cloud) → Generate Layout (cloud) → Generate with cables (cloud) → Generate with energy (cloud, Pro Plus key) → Cancel mid-flight (refund verified) → Failed run path (refund verified) → all 3 exports download → Recents view + RunsList render correctly → multi-tab → license-key swap.

Prod: PRODUCTION SIDECAR DELETION — release-build off the deletion branch; Arun verifies the same full customer flow on prod with a real Pro Plus license against api.solarlayout.in. NO ROLLBACK PATH after this row; captured in the decision memo with the verification evidence.

Out of scope
- Rolling back to the sidecar (forward-only).
- mvp_admin observability of cloud Lambdas (separate concern).

C20 — Mobile contract verification

Status: todo
Depends: C19
Tier: T1 (no new code; documentation row)

Goal
Verify and document that every desktop-facing contract is form-factor-agnostic. The mobile (Expo/RN) team — when they exist — should be able to consume the same API + auth + polling + download surfaces with no backend changes.

Locked
D1

Open verifications
- Walk every endpoint exercised by the desktop end-to-end and confirm: license-key bearer auth (not browser cookies), no Tauri-specific headers, no mailto-style URLs that don't work on mobile.
- Confirm S3 presigned URLs work for the mobile fetch layer (no IAM-signed-host weirdness).
- Confirm KMZ upload (B6 → S3 PUT) is doable from a mobile HTTP client (presigned URL is the abstraction; it's fine).

Acceptance
- docs/initiatives/findings/YYYY-MM-DD-NNN-mobile-contract-parity.md documents the endpoint matrix.
- No code changes in this row.

```
Out of scope
  - Building the mobile app.
  - Mobile-specific UX considerations.
```

C21 — Production cutover + V1 retirement signoff

```
Status:   todo
Depends:  C19
Tier:     T3 (decision memo)

Goal
  Final production cutover: every Run on prod runs through the
  cloud path. V1 endpoints (the legacy desktop install's
  `EntitlementSummary` etc.) can now be retired per V2-plan §2's
  "Legacy retirement criterion."

Locked
  none new

Open verifications
  - Confirm prod has been on cloud-paths for ≥2 weeks with DLQ
    empty and reconciler not catching anything pathological.
  - Confirm Prasanta's legacy install user-journey is supported by
    the cloud path or that user has migrated.

Acceptance
  - V1 endpoint frozen-markers (B21) → "deleted" markers.
  - Marketing site download CTA reflects desktop-app-cloud-only.
  - Retirement memo at docs/initiatives/findings/.

Smoke trigger (Arun, per sec 11.2) — final cutover smoke
  Local:    skip — this is a prod-state row.
  Prod:     PRODUCTION FINAL SIGN-OFF — verify on prod that:
            (a) any remaining V1 endpoint usage from legacy
installs
            still works OR has been migrated cleanly (curl the V1
            endpoints; confirm 410 Gone on deletion-marked
routes);
            (b) marketing site CTA on solarlayout.in reflects the
            new state;
            (c) DLQ has been empty for the prior 2 weeks
            (CloudWatch metric);
            (d) reconciler logs show no pathological patterns
            (mvp_api / Vercel Cron logs);
            (e) Prasanta's legacy install path either migrated or
            documented as support-only.
            Sign-off captured in retirement memo.

Out of scope
  - Deleting the V1 endpoints themselves (separate row, post-
launch).
```

10. Future Ladder (v2 / v3 — explicitly NOT BUILT)

These are documented to keep current decisions honest about future-proofing, NOT as backlog. Add only when telemetry proves the constraint.

v2 — functional split. Energy yield separates from compute-layout into its own Lambda + SQS queue. Triggered when a Run has `billedFeatureKey ∈ {energy_yield, generation_estimates}`. Layout completes first; energy publishes a follow-on message; rollup waits for both before flipping DONE. Adds: `compute-energy/` Lambda dir; `pvlayout-compute-energy-jobs` queue; rollup logic. Justification trigger: Lambda cost tells us energy network-wait is wasting CPU minutes, OR energy-only re-runs become a feature.

v3 — per-plot fan-out. Add `Slice` table (one row per boundary in a multi-plot Run). mvp_api publishes N SQS messages. Each Lambda handles one Slice. Rollup: when all Slices terminal, flip Run.status. Adds: `Slice` table + semantic-ID prefix; idempotent slice-update SQL becomes per-slice; result aggregation across N S3 blobs into one combined response; per-slice progress UI option re-enabled. Justification trigger: a real customer hits Lambda 15-min timeout, or worst-plot wall-clock > acceptable user-wait.

Escape hatches (no row, no work): - ECS Fargate via `CMD` change on the same Docker image (D22). For Runs estimated > 14 minutes. - Provisioned concurrency on first-of-day cold-start mitigation if metrics demand. - Outbox pattern instead of publish-then-commit if SQS reliability ever surprises us.

11. Verifications & Smoke-Testing Protocol

11.1 Cross-cutting one-time verifications (before C-rows)

These need a green checkmark before the rows that depend on them start. Recommend doing them all in a single short verification session at the top of C1 / C2 work.

#	Check	Required by	Where
V1	Vercel function timeout for `apps/mvp_api` (Hobby = 10s, Pro = 60s, Enterprise = 300s). PDF generation in compute-layout runs as Lambda not mvp_api so this only matters for sync invokes (parse-kmz, exports endpoint). 60s ample if Pro.	C4, C7, C17	`vercel.json` or Vercel dashboard
V2		C3	

#	Check	Required by	Where
	OIDC role for ECR push from GitHub Actions		`.github/workflows/`
V3	RDS connection from Lambda — VPC config or public-via-creds	C5, C6, C8	aws lambda get-function-configuration on existing dormant fn if any
V4	IAM user `renewable-energy-app` has SQS/Lambda/SNS perms in addition to S3	C5, C8	IAM console / inline policy
V5	mvp_api package.json has neither `@aws-sdk/client-sqs` nor `@aws-sdk/client-lambda` (likely both missing)	C4, C7	grep
V6	`Run.status` column type permits `QUEUED` (string column → yes; enum → migration needed)	C7	schema.prisma
V7	Vercel Cron availability on the project's plan	C13	Vercel dashboard

11.2 Smoke-testing protocol

> **Operating principle:** unit tests test small pieces of code; smoke tests test what the user experiences. **No "works on my laptop" claim is acceptable as evidence the row is done.** CI/CD is a first-class citizen and ships alongside code, not as an afterthought. *But* smoke testing has real human cost — Arun's time is the bottleneck — so the cadence is **strategic, not exhaustive**.

Today's deployment topology (as of v1.1 — 2026-05-03):

The original spec assumed a full Vercel + AWS staging deploy paralleling production. That topology does not exist today. Reality:

- **Production is the de-facto staging environment for app-layer changes** until external launch. The public release has not happened; only Arun and Prasanta exercise prod. CI/CD (GitHub Actions) deploys `mvp_api`, `mvp_web`, `mvp_admin` only to production — there are no Vercel staging projects. Smoke for app-code rows therefore happens on prod by definition.

- **Production stability bar is high regardless** — Stripe is live and wired to `api.solarlayout.in/webhooks/stripe`; RDS holds real entitlement state; S3 buckets accumulate real artifacts. Treat as production: don't break it; don't trigger live-Stripe charges that aren't intended; don't pollute shared state. Prod is **co-founder dogfood**, not a developer scratch space — when external customers land, the same prod environment serves them.
- **Staging is partial** — it covers the surfaces where mistakes are non-trivial to undo:
  - **Staging RDS** (`MVP_DATABASE_URL` in `.env.staging`) — every Prisma migration runs there first via the `set -a; . ./.env.staging; set +a; bunx prisma migrate ...` pattern. Production RDS rollback is non-trivial and disruptive; this gate is non-negotiable for any row that touches `schema.prisma`.
  - **Staging AWS resources** — `solarlayout-staging-{downloads,projects}` S3 buckets, IAM policies, eventually the SQS queues from C5 and Lambda functions from C4 / C6 / C8 / C16. Provisioning, permissions, and policy changes get exercised in staging AWS first to gain confidence before the prod analogue is created or modified.
  - **Staging Vercel apps DO NOT exist.** No staging deploy of `mvp_api`, `mvp_web`, or `mvp_admin`. App-code smoke happens at Local then directly at Prod.

Levels (reality-aware, ordered by cost):

Level	Environment	When	Who
Local	Laptop: sidecar (until C19) + mvp_api dev server + dev/local S3 (or LocalStack) + local Postgres or staging-RDS connection + LocalStack/real Lambda invoke	Every (smoke)-marked row, first	Arun
Staging	**Partial.** RDS staging DB for migrations; staging AWS resources (S3 / IAM / SQS / Lambda) for provisioning + permission verification. **No Vercel apps in staging.**	Rows that touch DB migrations (C7, C13) or AWS Lambda / SQS / IAM provisioning (C4, C8, C16, C18) — apply this level. App-code-only rows skip directly from Local to Prod.	Arun
Production	Live prod deploy (CI/CD wired here). Stripe live. Co-founders only until external	Every (smoke)-marked row, after Local + CI green. Replaces what the original spec	Arun

Level	Environment	When	Who
	launch. **De-facto staging for app code** in the meantime.	called "Staging" for app-layer smoke.	

Operator default: Arun for desktop UX + most rows. Prasanta when solar-domain correctness is the validation question (e.g., golden-test divergence on a real customer KMZ).

Cadence per (smoke) row — the 5-step Definition of Done:

1. **Automated gates pass** — pre-commit gate from CLAUDE.md §8 (`bun run lint && bun run typecheck && bun run test && bun run build`) plus any row-specific test gates.
2. **Local smoke (Arun)** — execute the row's `Smoke trigger` *Local* steps. **Bite-sized chunks** per memory's feedback rule: one observation per prompt, never bundle a numbered checklist. Capture output evidence in the row's PR description (paste of relevant tool output, DB query result, S3 ListObjects, mvp_api response).
3. **CI/CD green** — the row's PR is green on `main`. CD deploys to **production** automatically (no staging deploy step exists for Vercel apps). For DB-migration rows, CD also runs the migration against the configured DB target — verify migration status before merging.
4. **Staging smoke (Arun) — only if applicable.** Execute the row's `Smoke trigger` *Staging* steps **only** if the row declares one — that's DB-migration rows (C7, C13) and AWS-resource rows (C4, C8, C16, C18) per the levels table above. Otherwise skip — proceed to step 5.
5. **Production smoke (Arun) — the actual smoke gate today.** Execute the row's `Smoke trigger` *Prod* steps (or its *Staging* steps re-targeted at prod for app-code rows). Same bite-sized cadence. **Don't break Prod.** Don't trigger live-Stripe charges that weren't intended (the entitlement system reads the live-billed Subscription chain). For phase-end "PRODUCTION CUTOVER" rows (C9, C19, C21), this step also doubles as the cutover sign-off captured in the row's decision memo — those rows still represent first-use-on-prod milestones for new compute paths and warrant the heavier ceremony.

Rows that are NOT (smoke)-marked: automated gates + code review is enough. CI/CD still runs against production on every merge, but no manual sign-off is needed. Examples: doc cleanup, refactor-only, scaffold-only, documentation-only rows.

Anti-patterns to avoid:

- ❌ Bundling smoke checks into a numbered list ("steps 1-7"). One question per prompt; wait for human response. Memory feedback rule.
- ❌ Claiming a row done before its applicable-level smokes pass. "Works on my laptop" is not done.
- ❌ Smoking every row "to be safe." Smokes for pure refactors waste Arun's time and dilute the signal of the genuinely high-risk smokes.
- ❌ Skipping local smoke and going straight to prod. Local catches 80% of issues at 1× the cost; prod issues are more expensive to debug AND visible to shared state (Stripe, RDS, S3).

- ❌ Treating Prod as a developer testbed. Co-founder dogfood ≠ developer scratch — Stripe charges are real, S3 objects accumulate cost, data integrity for the first paying customers depends on what lands here.
- ❌ Skipping staging-RDS migration smoke for migration rows. Production RDS rollback is non-trivial; this gate exists to prevent that.

Smoke evidence captured in the PR:

```
## Smoke evidence (row Cnn)

### Local
- <observation 1>: <evidence — output paste, screenshot, etc.>
- <observation 2>: <evidence>
...

### Staging  (only for DB migration / AWS provisioning / IAM rows)
- <observation 1>: <evidence>
...

### Prod
- <observation>: <evidence>
```

Each `Smoke trigger` field in row §9 names the specific exercises for that row, written reality-aware (v1.1) — Staging entries appear only on rows that touch DB migrations (C7, C13) or AWS-resource provisioning (C4, C8, C16, C18) and target staging RDS / staging AWS specifically; app-code rows have Local + Prod entries only. The protocol above governs **what / when / who**; **§11.4 governs HOW to run a smoke session** (ST-ID nomenclature, pre-req validation, bite-sized step execution, P0–P3 finding classification, outcome protocol, dual evidence capture). Smoke sessions are launched via the **§13.2 cold-session prompt** — a fresh Claude Code session dedicated to driving one ST-ID end-to-end.

11.3 Brainstorm-first vs writing-plans-direct

> **Operating principle:** rows that have ANY undecided design or UX question — even a small one — produce dramatically better outcomes when the child session invokes `superpowers:brainstorming` *before* `superpowers:writing-plans`. This is structural, not stylistic.

Why structurally:

- **`superpowers:brainstorming`** has a HARD-GATE in its skill body: *"Do NOT invoke any implementation skill until you have presented a design and the user has approved it."* The skill enforces: explore project context → ask clarifying questions **one at a time** → propose 2-3 approaches with trade-offs → present design in sections, user approves each → write spec → user reviews spec → ONLY THEN handoff. Dialogue is mandatory; the skill cannot proceed past gates without user input.

- **`superpowers:writing-plans`** is the implementation translator. Its job is to convert an *approved* design into an executable plan. When given an underspecified row, it fills the gaps with its own reasonable-looking decisions

— silently — because that is the correct role of a plan-translator skill. It is not broken; it is working as designed. But it is the **wrong tool** when the design itself is incomplete.

Practical consequence (learned the hard way): writing-plans pattern-matches the spec template and produces output. Brainstorming reads, analyzes, asks. The halted B32 plan that prompted this whole architectural reset was a writing-plans output against a row whose architecture hadn't been brainstormed — and it confidently produced ~150 LOC of detailed implementation around an architecture that didn't exist.

Spec convention:

Rows that need brainstorming-first carry a `Brainstorm-first: yes` field with a one-line Reason. **Absence of the field = writing-plans-direct is acceptable.** The cold-session prompt (§13) enforces this — when the row says yes, the child session MUST invoke `superpowers:brainstorming` first; the brainstorm output is committed at `docs/superpowers/specs/YYYY-MM-DD-<row-id>-<slug>.md` *before* `superpowers:writing-plans` is invoked. **The judgment call is removed from the cold session — the spec declares it.**

Decision criteria for marking a row YES: - Open design or UX questions whose answer materially changes the implementation. - First instance of a new pattern (e.g., first Lambda → first SQS Lambda → first cutover). - Irreversibility risk (e.g., sidecar deletion, V1 retirement). - Multi-component interaction where ordering or contract is non-obvious. - Recent code reality may have diverged from a spec assumption.

Decision criteria for marking NO (or omitting): - Pure refactor (file moves with green tests). - Pure scaffold (template-driven structure). - Documentation-only row. - Mechanical extension of an established pattern (mirrors a working sibling row verbatim). - Row body fully specifies handler shape / SQL / wire contract — no design questions remain.

About half the rows in §9 are marked YES. That's not over-engineering — it reflects how much real design surface this arc actually contains.

11.4 Smoke session execution protocol

> **Smoke tests are formal AI-supported, human-executed tasks** with clear pre-requisites, step-by-step instructions, priority-classified outcomes, and a documented protocol for addressing findings. They are NEVER bundled into the row's implementation session — they happen on a separate cold Claude Code session whose ONLY job is to drive the smoke. The §13.2 prompt template launches such a session.

ST-ID nomenclature

Every smoke test instance is identified as `ST-<row-id>-<level>` where `<level>` `∈ {L, S, P}` for Local / Staging / Prod. Examples:

- `ST-C4-L` — local smoke for parse-kmz Lambda row
- `ST-C9-S` — staging smoke for desktop migration row
- `ST-C19-P` — prod smoke for sidecar deletion (the irreversibility one)

Sub-steps within a session are referenced as `ST-C9-S.1`, `ST-C9-S.2`, etc. when needed — typically when capturing findings ("P1 found at ST-C9-S.4").

Session shape (the arc every smoke session follows):

1. **Pre-requisite validation FIRST.** The session verifies, before any smoke step:

   - The target row in §9 has `Status: in-progress` (not `done` — smoke is the gate that flips done; not `todo` — implementation must be in flight).
   - All of the row's `Depends` rows have `Status: done` in §9.
   - The target environment is live and reachable for the chosen level (per §11.2 reality):
     - **Local (L)**: dev server up (`bun run dev` showing `mvp_api ready`); local DB connectable; sidecar (until C19) responding to `/health`.
     - **Staging (S) — partial-only**: applicable only when the row's `Smoke trigger` field declares a Staging entry — that's DB-migration rows (C7, C13) and AWS-resource rows (C4, C8, C16, C18) per §11.2. For DB rows: staging RDS reachable via `set -a; . ./.env.staging; set +a; bunx prisma migrate status --schema=packages/mvp_db/prisma/schema.prisma`. For AWS rows: the specific staging AWS resource the row exercises is provisioned and reachable (queue exists, IAM policy attached, bucket reachable, Lambda function deployed, etc.). If a row has no Staging entry, skip directly to Prod.
     - **Prod (P) — the de-facto smoke gate today**: prod deploy green on the latest commit (`gh pr checks` or recent CD run); `api.solarlayout.in/health` returns 200; co-founder license key authenticates; the specific prod AWS resource the row exercises is provisioned.
   - No stale state from a prior smoke (no orphan Run rows in dev / staging / prod RDS, no leftover S3 objects under known fixture key prefixes, no stuck SQS messages).

   If any pre-req fails: **HALT.** Report what failed; smoke is rescheduled. Do NOT improvise a workaround.

2. **Bite-sized step execution.** Claude breaks the row's `Smoke trigger <level>` entry into atomic executable steps. **One step per prompt.** Each step is concrete and runnable:

   - ✅ "In another terminal at repo root, run `bun run dev` and report back when you see `mvp_api ready on :3003`."
   - ✅ "Open AWS console → Lambda → `pvlayout-compute-layout-staging` → Monitor tab. Paste the most recent invocation timestamp + duration."
   - ✅ "In Tauri, click the Generate button. Paste any console errors that appear in the next 10s."
   - ❌ "Verify mvp_api works." (abstract; no atomic action)
   - ❌ "Test all four status transitions." (multiple actions bundled)

   Human runs the step, pastes the **actual output** back. Claude analyzes and produces the next step. No numbered checklists, no batching. (Memory feedback rule.)

3. **Findings classification (P0–P3).** Each anomaly observed during smoke is classified by Claude with the human's confirmation:

Priority	Definition	Row impact
P0	Blocker. Basic functional path doesn't work. Customer flow broken. Data integrity compromised.	Row CANNOT flip to done. Smoke halts; fix is mandatory before re-smoking.
P1	Significant. Happy path works but a real bug, missing UX state, or design flaw emerges.	Row CANNOT flip to done. Either inline-fix (same PR) or create a new row this row depends on.
P2	Minor. Cosmetic, edge-case, nice-to-have. UX rough edge that doesn't break the flow.	Row CAN flip to done. Captured as a follow-up — appended to a not-yet-started downstream row's notes, or a new low-priority row.
P3	Observation only. Worth recording for posterity; documents the state of the system at smoke time.	No action. Recorded in SMOKE-LOG.md only.

1. **Outcome protocol per priority:**
   - **P0** → fix INLINE in the row's branch + same PR. Re-run smoke from the failing step (not from scratch — Claude resumes from where the failure occurred). Row stays in-progress.
   - **P1** → if scope is **small + bounded** (≤ half-day work, no new files, no design re-think): fix INLINE in same PR; re-smoke. Otherwise: **create a new row** via spec amendment commit (per §12) — typically inserted with a sub-row notation (e.g., `C8.1` between `C8` and `C9`) or appended as a new top-level row at the end of the relevant phase. The originating row's `Depends` updates to include the new row; row stays in-progress until the new row also closes.
   - **P2** → captured as follow-up. Either appended to a not-yet-started downstream row's notes (preferred when fit is natural), OR a new low-priority row at the end of the relevant phase. Originating row CAN flip to done.
   - **P3** → recorded in `docs/post-parity/SMOKE-LOG.md`. No action.

2. **Evidence capture — TWO places, both required:**
   - **The row's PR description** carries the immediate session log under a `## Smoke evidence (ST-<id>)` section per §11.2's evidence template. This is the per-PR record.
   - **`docs/post-parity/SMOKE-LOG.md`** carries a cross-cutting entry per ST-ID: date, operator, pre-req validation result, atomic-step results (one line per ST-ID.N step), findings classified P0–P3, outcomes (inline fix? new row? bundled into XYZ?), session duration. **This file is the searchable archive**

of every smoke ever run — protects future cold sessions from re-discovering the same bug.

3. **Spec amendments from smoke findings.** P0 / P1 findings that surface a flaw in a locked decision (D-id) trigger a spec amendment commit per §12 BEFORE the row's implementation fix lands. The amendment commit names the D-id changed and why; the row's fix commit references the amendment. **Never silently work around a locked decision.**

What NOT to do during a smoke session:

- ❌ Run a smoke session from inside an implementation session. Different cognitive shapes; mixing produces sloppy smoke and rushed implementation. Use a fresh Claude Code session per §13.2.
- ❌ Bundle steps. One step per prompt. Period.
- ❌ Mark a row done with unresolved P0 or P1 findings. The smoke gate is real.
- ❌ Skip pre-requisite validation. "It probably works" is how prod fires happen.
- ❌ Capture evidence only in the PR description without updating `SMOKE-LOG.md`. The cross-cutting archive is what protects future cold sessions from re-discovering the same bug.
- ❌ Improvise around an environment problem mid-smoke. If staging is half-broken, halt and surface; don't smoke through it and call results valid.

11.5 Post-row completion protocol

> **Operating principle:** the row's spec text, plan, and live execution can drift apart. Drift caught BEFORE the row closes is cheap to fix; drift left in the spec rots into surprises for the next cold session. Every row close runs this protocol — a brief structured retrospective that surfaces drift candidates and asks for concurrence before any spec amendment lands. It is NOT a full design re-think; it is an "anything I learned that the next cold-session reader needs?" pass.

When this protocol fires: AFTER the row's Acceptance criteria are met + all applicable smoke gates passed (per §11.2's 5-step DoD), BEFORE flipping `Status: in-progress → done`. Cold-session implementation runs (per §13.1) end with this step — it's wired into the §13.1 prompt's operating rule #5.

What this protocol does NOT cover (these are mechanical per §12 and don't need discussion):

- Status flip `in-progress → done` itself.
- `Plan:` and `Shipped:` cross-reference lines.
- The atomic row-close commit message.

Four-category drift check — run silently first; surface only what's material:

1. **Stale assumptions in row text.** A fact stated in the row body (Goal / Open verifications / Acceptance / Out of scope / Smoke trigger) that turned out wrong during execution. Example: a recon list of N affected files where execution found 2N. Recommend: amend row text to reflect reality, OR amend the convention if the gap recurs across rows.
2. **New rows surfaced by execution.** Discoveries that cleanly belong in their own future row (not inline-fixable per the §11.4 P0/P1 outcome protocol). Example:

a smoke finding that a downstream row's preconditions need a precursor cleanup row. Recommend: the new row's position in §9 (which phase, which Depends).
3. **Adjacent-row scope gaps.** A discovery that a not-yet-started downstream row's text is incomplete or now wrong given what landed. Example: a fixture relocation in this row implies a future row's inventory step needs a corresponding update. Recommend: specific edit + which row(s) it touches.
4. **Locked-decision (D-id) implications.** Rare. A row's execution surfaces evidence that a locked decision is wrong / unworkable / needs nuance. Per §12 this requires a dedicated spec amendment commit BEFORE the row's implementation lands; if discovered at row close (rather than mid-execution), it BLOCKS the close — escalate to Arun, do not flip the status until the D-id amendment is decided.

Trigger condition (when to actually start the discussion vs skip):

Run the four-category check silently. If all four yield "nothing to surface" — proceed with the mechanical close per §12. Common case for pure-refactor or scaffold rows where the plan + execution matched the spec exactly. The protocol's pass-through behavior is the default; only surface when something material was found.

If anything surfaces — discussion shape (one item at a time):

Apply the memory-feedback "one question at a time during design / brainstorming" rule. Per surfaced item:

1. Agent presents the finding in plain prose: what category, what was observed during execution, what the candidate spec amendment would look like.
2. Agent recommends an action with reason: amend row text / add new row / leave as tradecraft (not spec content) / escalate D-id.
3. Arun responds: concur / amend recommendation / reject.
4. If concurred: the amendment commit lands per §12 (dedicated commit, version bump in §15 changelog) BEFORE the row-close commit.
5. Move to the next surfaced item.

Negative outcomes are common and correct. "I noticed X but it's tradecraft, not spec content — leave it" is a valid conclusion. "I noticed X but the spec's existing language already covers this in spirit" is a valid conclusion. Don't manufacture amendments to look productive.

Outputs:

- Per surfaced item: zero-or-more spec-amendment commits (per §12), each appending a row to the §15 changelog.
- A brief mention in the row's PR description under a `## Post-row completion protocol` section, listing the four categories + their dispositions. Even "nothing material surfaced" gets recorded so future readers know the protocol ran.

11.6 Pre-row-start protocol

> **Operating principle:** every row begins on a clean slate. A fresh cold session starting work without verifying its starting state risks landing the row's first WIP commit on top of stale code, a wrong branch, or unmerged work — mistakes that are expensive to unwind. The pre-row-start protocol is the bookend to §11.5: §11.5 closes one row cleanly; §11.6 opens the next one cleanly.

When this protocol fires: at the start of a cold-session implementation run (per §13.1), BEFORE any code action, BEFORE invoking `superpowers:brainstorming` or `superpowers:writing-plans`, BEFORE the first WIP commit. It is the FIRST mutating-action gate of every row.

Pre-conditions to verify (in order):

1. **Prior row's PR merged** — Arun confirms in the §13.1 cold-session prompt by filling the "Prior row's PR" slot with a URL or "merged at ". If empty or "no" — HALT and ask. (For the very first row in the arc: "N/A — first row".)

2. **`git fetch origin`** succeeds (network + auth + remote reachable).

3. **Local `main` matches `origin/main`** — no divergence:

   ```
   [ "$(git rev-parse main)" = "$(git rev-parse origin/main)" ]
   ```

4. **Working tree clean** — `git status --porcelain` is empty.

5. **Current branch is `main`** — `git branch --show-current` returns `main`.

6. **Target row's `Status` is `todo`** in §9 — not `in-progress` (another session may be running it; or the prior session didn't close cleanly), not `done` (already shipped), not `blocked` (resume conditions need addressing first).

7. **Target row's `Depends` rows all have Status `done`** in §9 — dependency chain is satisfied.

Required steps once all 7 pre-conditions pass:

```
# 1. Ensure on main + clean
git checkout main
git status --porcelain   # must be empty

# 2. Pull latest from origin (fast-forward only — refuses
        divergence)
git fetch origin
git pull --ff-only origin main

# 3. Branch off the freshly-pulled main
git checkout -b <branch-name>
```

Branch naming convention:

Use conventional-commit type as the prefix:

- `chore/c<NN>-<short-slug>` — refactor, structural, dep update (e.g., `chore/c2-extract-pvlayout-core`, `chore/c19-delete-sidecar`).
- `feat/c<NN>-<short-slug>` — new functionality (e.g., `feat/c4-parse-kmz-lambda`, `feat/c8-compute-layout-end-to-end`).
- `fix/c<NN>-<short-slug>` — fix-shaped row (rare; rows usually create not fix).
- `docs/spec-v<X.Y>-<short-slug>` — spec amendment commits per §12 (e.g., `docs/spec-v1.1-protocol-amendments`). Spec amendments may also commit direct-to-main when Arun explicitly authorizes a low-risk docs-only change.

On ANY pre-condition failure: HALT.

Failure	Surface to Arun
Prior PR not confirmed merged	Ask explicitly. Do not proceed on assumption.
`git fetch origin` failed	Network / auth / remote-reachability error. Paste the error. Wait.
`git pull --ff-only` rejected (divergent local main)	Local main has unexpected commits not on origin. Paste `git log origin/main..main`. Ask: stash, discard, force-push origin, or investigate.
Working tree not clean	Paste `git status` output. Ask: commit, stash, discard, or investigate.
Current branch not `main`	Paste current branch + `git status`. Ask: commit, stash, switch, or investigate.
Target row Status not `todo`	Paste the row's current Status line. If `in-progress`: another session may be active or prior session left it open. If `done`: row already shipped. If `blocked`: surface the blocker note.
Target row's `Depends` not all `done`	List the missing dependencies. Wait — those rows must close first.

Why HALT instead of auto-recover:

Local-state discrepancies usually mean either (a) the previous session didn't close cleanly, or (b) Arun made a manual change the agent shouldn't silently overwrite. Auto-recovery (`git stash`, `git reset --hard`, `git checkout -- .`) risks destroying real work. **Always surface; never silently mutate state.**

What happens AFTER pre-flight passes:

The new branch is created. Then the row's normal §13.1 operating rules take over: brainstorming-first if marked, writing-plans, executing-plans, smoke gates, post-row completion protocol (§11.5), commit, PR.

Cold-session integration:

Wired into §13.1 as operating rule #0 — runs BEFORE rules 1–5. The cold-session prompt template makes this explicit so a fresh session can't accidentally start work on a dirty or stale state.

12. Tracking Protocol

This spec is a living document. The Implementation Rows section (§9) is the source of truth for what's done, in-progress, todo, blocked across the entire cloud-offload arc.

Status states: - `todo` — not yet started. - `in-progress` — a session is actively working it. - `done` — merged to main; live-verified where applicable. - `blocked` — explicitly halted; row notes name the blocker.

Update cadence: - Row claimed: `todo → in-progress` in same commit as first WIP. - Row completed: run the §11.5 post-row completion protocol; if it surfaces material drift, the corresponding spec amendment commits land first; THEN `in-progress → done` in same commit as merge (or final row commit). - Row blocked: `in-progress → blocked` immediately on discovery; notes name blocker + resume conditions.

Cross-references appended to a row on close:

```
Plan:     docs/superpowers/plans/YYYY-MM-DD-<row-id>-<slug>.md
Shipped:  <PR URL or commit SHA>
```

Spec amendments: If a child session discovers a locked decision (D-id) is wrong — architectural reality has changed, or a decision turned out to be unworkable — the spec is amended in a dedicated commit BEFORE the row's implementation commit. The amendment commit message names the D-id changed and why. **Never silently work around a locked decision.**

Atomic commit per row (matches `docs/PLAN.md` convention).

13. Cold-Session Prompt Templates

Two distinct cold-session prompts: §13.1 for **implementation** sessions (executing a row), §13.2 for **smoke-test** sessions (running an ST-ID per §11.4). They are deliberately separate — implementation and smoke have different cognitive shapes; mixing them produces sloppy smoke and rushed implementation.

13.1 Implementation cold-session prompt

Paste verbatim into a fresh Claude Code session. Replace `<ROW-ID>` and `<ROW-NAME>` with the target.

```
We are continuing the cloud-offload-compute architecture project.
This is a cold Claude Code session; you have NO prior context.

Before doing ANYTHING ELSE — including clarifying questions — read
this spec end-to-end:
```

```
  docs/superpowers/specs/2026-05-03-cloud-offload-architecture.md

This spec is the primary source of truth for the entire cloud-
offload
arc. It contains numbered locked decisions (D1–D23) and numbered
implementation rows (C1–C21). I will reference IDs throughout our
session.

I want to execute row: <ROW-ID>: <ROW-NAME>
Prior row's PR (or N/A if first row): <PR URL, "merged at <SHA>",
or "N/A — first row" — Arun fills>

Operating rules:

0. PRE-ROW-START PROTOCOL (§11.6). Before any code action, before
   invoking brainstorming or writing-plans, run the §11.6 pre-
flight
   gate: verify the prior PR confirmation above, `git fetch
origin`,
   local `main` matches `origin/main`, working tree clean, current
   branch is `main`, target row's Status is `todo`, all of the
   row's `Depends` rows have Status `done`. If all 7 pass: create
   the row's feature branch off the freshly-pulled `main` per
   §11.6's branch naming convention; that's where this row's work
   lands. On ANY discrepancy: HALT and surface to me — auto-
recovery
   is not allowed (risk of destroying real work).

1. Locked decisions are non-negotiable. If you think one is wrong,
   STOP and surface to me explicitly — do not work around
silently.
   We amend the spec via a dedicated commit; that's a separate
   conversation.

2. Code reality wins over spec. Run every check in the row's "Open
   verifications" against the current codebase before proceeding.
   The codebase has likely moved since the spec was written; if
   reality contradicts a spec assumption materially, surface the
   conflict before continuing.

3. Stay inside the row's Goal + Acceptance + Out of scope. Do not
   silently expand.

4. Use superpowers in the order the row dictates — DO NOT EXERCISE
   JUDGMENT here, the spec already decided:

   - **If the row has `Brainstorm-first: yes`:** you MUST invoke
     `superpowers:brainstorming` FIRST. Commit the brainstorm
output
     at `docs/superpowers/specs/YYYY-MM-DD-<row-id>-<slug>.md` and
     get my approval BEFORE invoking writing-plans. The
     `Brainstorm-first` field's `Reason` line names what you must
     surface in dialogue. See sec 11.3 for why.
```

```
   - **If the row has NO `Brainstorm-first` field:** invoke
     `superpowers:writing-plans` directly → my sign-off →
     `superpowers:executing-plans` (TDD). The row body fully
     specifies the design; writing-plans-direct is acceptable.
   - **Either way:** the standard pre-commit gate from CLAUDE.md
     sec 8 applies, plus any row-specific test gates listed under
     Acceptance.

   Why this is non-negotiable per row: writing-plans is mechanical
   by design. When given an underspecified row, it fills the gaps
   silently with its own decisions — confidently and wrongly. The
   halted B32 plan that prompted this whole architectural reset
   was a writing-plans output against a row whose architecture
   hadn't been brainstormed. Don't repeat that.

5. On completion: BEFORE flipping the row's Status from
   todo/in-progress → done, run the §11.5 post-row completion
   protocol — a brief four-category retrospective for spec drift
   (stale assumptions / new rows / adjacent scope gaps / D-id
   implications). If anything material surfaces, discuss with me
   one item at a time and land any concurred spec amendments per
   §12 (dedicated commit + §15 changelog bump) BEFORE the row
   close. Then flip Status to done, append `Plan:` and `Shipped:`
   lines, commit as part of the row's atomic commit.

Begin by:
  (a) Reading the full spec.
  (b) Running the §11.6 pre-row-start protocol — return either a
      HALT report (with what failed) or a "pre-flight OK; created
      branch <name>" confirmation before going further.
  (c) Reading the row's own entry (§9), the "Open verifications"
      surfaces in current code, and confirming all "Depends" rows
      are marked done in §9.

Then propose your plan to me before writing any code.
```

13.2 Smoke-test cold-session prompt

Paste verbatim into a fresh Claude Code session — separate from any implementation session. Replace `<ROW-ID>` with the target row and `<L|S|P>` with the smoke level.

```
We are running a smoke test for the cloud-offload-compute
architecture project. This is a cold Claude Code session; you have
NO prior context. Smoke-test sessions are dedicated tasks separate
from implementation work — your ONLY job is to drive this smoke.

Before doing ANYTHING ELSE — including clarifying questions —
read:

  1. docs/superpowers/specs/2026-05-03-cloud-offload-
architecture.md
     - sec 11.2 (smoke testing protocol — what/when/who)
```

```
     - sec 11.4 (smoke session execution — ST-IDs, priorities,
       outcome protocol; this is your operating manual)
     - The target row's full entry in sec 9, especially its
       `Smoke trigger` field
  2. docs/post-parity/SMOKE-LOG.md (the prior smoke history for
     context — what was tested before, what findings emerged)

I want to execute smoke test: ST-<ROW-ID>-<L|S|P>

(L = Local laptop env; S = Staging deploy; P = Production. The
row's
`Smoke trigger` field has a per-level entry naming the exercises.)

Operating rules — DO NOT EXERCISE JUDGMENT, the spec decided:

1. PRE-REQUISITE VALIDATION FIRST. Before any smoke step, verify:
   - The target row in spec sec 9 has `Status: in-progress`
     (not `done`, not `todo`).
   - All of the row's `Depends` rows have `Status: done` in sec 9.
   - The target environment is live and reachable for the chosen
     <L|S|P> level. Run concrete checks (curl, AWS CLI, browser
     load) and report back.
   - No stale state from prior smokes (orphan rows in dev RDS,
     leftover S3 objects, stuck SQS messages).
   If ANY pre-req fails: HALT. Tell me what failed; we reschedule
   the smoke. Do not improvise around environment problems.

2. BITE-SIZED STEPS. Break the row's `Smoke trigger` <level>
   entry into atomic executable steps. ONE STEP PER PROMPT.
   Each step must be concrete and runnable:
     ✅ "In another terminal at repo root, run `bun run dev` and
         report when you see `mvp_api ready on :3003`."
     ✅ "Open AWS console → Lambda → pvlayout-compute-layout-
staging
         → Monitor tab. Paste the most recent invocation timestamp
         + duration."
     ❌ "Verify mvp_api works." (abstract)
     ❌ "Test all four status transitions." (multiple actions
         bundled)
   I will run the step and paste actual output back. You analyze,
   then send the next step. No numbered checklists. No batching.

3. CLASSIFY FINDINGS as we go. Each anomaly = P0 / P1 / P2 / P3
   per spec sec 11.4. State your classification + reason; I
confirm
   or adjust:
     - P0 = blocker (basic flow broken, data integrity)
     - P1 = significant (real bug, missing UX state, design flaw)
     - P2 = minor (cosmetic, edge-case)
     - P3 = observation only

4. APPLY OUTCOME PROTOCOL per spec sec 11.4:
     - P0 → halt smoke; fix INLINE in the row's branch + same PR;
       re-run from the failing step (not from scratch).
```

```
     - P1 → if scope is small + bounded (≤half-day, no new files,
       no design re-think): fix INLINE same PR. Otherwise: create
       a new row via spec amendment commit per sec 12; row stays
       in-progress until the new row also closes.
     - P2 → captured as follow-up; appended to a not-yet-started
       downstream row's notes OR a new low-priority row at the
       end of the relevant phase. Originating row CAN flip done.
     - P3 → recorded in SMOKE-LOG.md only. No action.

5. CAPTURE EVIDENCE in TWO places, both required:
     - The row's PR description (under `## Smoke evidence
       (ST-<id>)`) — the per-PR record.
     - docs/post-parity/SMOKE-LOG.md (cross-cutting archive entry
       per ST-ID with date, operator, pre-req result, atomic-step
       results, findings classified P0–P3, outcomes, session
       duration).

6. SPEC AMENDMENTS. If a finding contradicts a locked decision
   (D-id), STOP and surface to me. Spec amendment commit per sec
12
   happens BEFORE any fix lands. Never silently work around a
   locked decision.

7. SESSION END. If all steps passed with at most P2/P3 findings
AND
   this smoke level is the gate the row needs (per sec 11.2's 5-
step
   DoD): tell me; I (Arun) flip the row's Status to `done` in spec
   sec 9 via an atomic commit. If unresolved P0/P1 remain: row
stays
   in-progress; we discuss next steps.

Begin by:
  (a) Reading sec 11.2, 11.4, and the target row's `Smoke trigger`
      field in sec 9.
  (b) Reading recent SMOKE-LOG.md entries for context.
  (c) Running pre-requisite validation as your first concrete
      action — return the result before anything else.
```

14. Doc Kill List

Disposition table for every existing doc that touches cloud offload, Spike 1/2, cable compute, run cancellation, or refund policy. Owned by row C1.

Path	Disposition	Rationale
`docs/superpowers/plans/2026-05-02-b32-failed-runs-path.md`	**DELETE**	Halted plan based on wrong architecture (Spike-2 internal-secret callback). Superseded by D9 + D17 + C12.

Path	Disposition	Rationale
`docs/post-parity/PRD-cable-compute-strategy.md`	**ANNOTATE SUPERSEDED** + link to this spec in header	Spike 1 shipped (preserve as historical context). Spike 2 architecture is fully replaced by this spec.
`docs/initiatives/2026-05-01-cable-compute-offload-feasibility.md`	**ANNOTATE SUPERSEDED** + link	Backend-side feasibility assessment; useful history; architecture replaced.
`docs/initiatives/findings/2026-05-02-002-refund-on-cancel-policy.md`	**AMEND §B.6**	Decisions A through B.5 stand. §B.6's "internal endpoint sidecar callback" framing is wrong — replace with reference to D9 + D17.
`docs/initiatives/post-parity-v2-backend-plan.md` rows B27-B34	**AMEND**	B27 stays as decision row. B28-B34 redirect to this spec's C-rows (B28 → C14; B29 done; B30 done; B31 → C11; B32 → C12; B33 → C10; B34 → C15).
`docs/PLAN.md` references to "renewable_energy session" / Spike 2	**AMEND**	Post-merge; redirect at this spec for cloud-offload work.
`docs/post-parity/RESUME-*.md`	**KEEP**	Historical session logs. Not load-bearing.
`docs/post-parity/SMOKE-LOG.md`	**KEEP**	Active session log.
`docs/post-parity/findings/2026-04-30-002-cable-perf-poc.md`	**KEEP**	Real perf data; informs §1 motivation.
`docs/post-parity/findings/2026-05-01-001-cable-perf-architecture-research.md`	**KEEP**	Research input; informs the architecture choices in this spec.
`docs/historical/*`	**KEEP**	Audit trail of superseded plans (CLAUDE.md §2 forbids modification).

15. Changelog

This section tracks amendments to the spec after the initial 2026-05-03 lockin. Each amendment is committed in a dedicated commit per §12 ("Spec amendments") and appends a row here. Per-row Status / `Shipped:` updates from §9 row closes are NOT changelog entries — they're tracked in the rows themselves; this changelog is for spec-level structural / decision / protocol amendments.

Version	Date	Sections touched	Change
v1	2026-05-03	Entire spec	Initial lockin. D1–D23 locked; C1–C21 row table established; smoke-testing protocol (§11.2 + §11.4) and brainstorm-first policy (§11.3) defined; cold-session prompts (§13) drafted; doc kill list (§14) executed.
v1.1	2026-05-03	Header status / version line, §11.2 (smoke-testing protocol — levels table + cadence + anti-patterns + evidence template + cleanup paragraph), §11.4 (pre-req validation step), §11.5 (NEW — post-row completion protocol), §11.6 (NEW — pre-row-start protocol), §12 (update cadence back-reference to §11.5), §13.1 (cold-session prompt — new rule #0 invokes §11.6 pre-flight; rule #5 amended to invoke §11.5; new "Prior row's PR" slot for Arun to fill; "Begin by" steps reordered to	Reality amendment: deployment topology + session-boundary protocols. The original spec assumed a full Vercel + AWS staging deploy paralleling production. As of v1.1, no Vercel staging exists — production is the de-facto staging environment for app-layer smoke (co-founder-only, Stripe live, no external customers yet). Staging is partial: RDS staging DB for migrations + staging AWS resources for provisioning / IAM verification. **All 15 (smoke)-**

Version	Date	Sections touched	Change
			marked rows in §9 had their `Smoke trigger` fields rewritten in-place to be reality-aware: 9 app-code rows dropped Staging entirely (Local + Prod only — C9, C10, C11, C12, C14, C15, C17, C19, C21); 2 DB-migration rows kept Staging as RDS-only (C7, C13); 4 AWS-resource rows kept Staging as AWS-only (C4, C8, C16, C18). Cleanup paragraphs in §11.2 / §11.4 simplified accordingly. **Two new session-boundary protocols added:** §11.5 post-row completion protocol — a four-category drift check (stale row text / new rows / adjacent scope gaps / D-id implications) that fires before every row's status flip and surfaces candidate spec amendments for Arun's concurrence. §11.6 pre-row-start protocol — a 7-precondition pre-flight gate (prior-PR-merged confirmation, `git fetch origin`, local
		put pre-flight before deep reading), §9 row Smoke trigger fields (15 in-place rewrites), this changelog (NEW §15).	

Version	Date	Sections touched	Change
			`main` matches `origin/main`, clean tree, on `main`, row Status `todo`, Depends all `done`) that fires at the start of every cold-session implementation run; halts on ANY discrepancy rather than auto-recovering (risk of destroying real work). Both protocols wired into the §13.1 cold-session prompt — rule #0 (§11.6 pre-flight) and rule #5 (§11.5 post-row close) bracket the row's lifecycle. **No locked decision (D-id) was changed**; this is a smoke-protocol + tracking-protocol reality amendment, not an architectural one.

Format for future amendments: append a row above. Bump the patch version (v1.2, v1.3, …) for protocol / structural changes that don't touch a D-id; bump the minor version (v2.0) for any amendment that changes a locked decision. Mention every section touched. Keep the `Change` cell to one paragraph; link to a longer memo at `docs/initiatives/findings/` if more detail is needed.

End of spec. This is the source of truth for cloud-offload compute architecture work. Implementation cold sessions: §13.1. Smoke-test cold sessions: §13.2. Live status of rows: §9. Decision IDs cited by rows: §3. Smoke-test protocol: §11.2 + §11.4. Amendment history: §15.